

Linux Kernel

Joe

March 9, 2026

目录

1	kernel 引导	1
1.1	kernel 构建	1
1.2	kernel 加载	1
1.2.1	jtag 加载	1
1.3	kernel 引导	1
1.3.1	booti 方式	1
1.3.2	bootz 方式	1
1.3.3	ram 文件系统	1
2	kernel 调试	2
3	kernel 内存划分	3
4	kernel 生成过程	4
5	代码分析	7
5.1	内核引导过程	7
5.2	zImage 入口函数	8
6	tools	34
7	SOC 架构	34
7.1	MMU	35

表格目录

1	vmlinux 主要部件	6
2	Config 前端	34

Listings

1 kernel 引导

1.1 kernel 构建

kernel 构建, 一般 3 大步. 构建完成后,Linux 内核会在 arch/arm(或 arm64)/boot 目录下生成 Image(zImage/uImage) 镜像文件, 如果使用设备树的话还会在 arch/arm(或 arm64)/boot/dts 目录下生成开发板对应的.dtb(设备树) 文件.

- 清理环境:make clean
- 配置参数:make xxx_defconfig
- 构建: make -j4

1.2 kernel 加载

1.2.1 jtag 加载

在 gdb 中用 jtag 方式将构建出来的二进制 Image 加载到 ddr.

```
# in gdb
restore ~/hwd/hwd10909_a53/linux/build/arch/arm64/boot/Image                binary 0x08400000
restore ~/hwd/hwd10909_a53/linux/build/arch/arm64/boot/dts/hwd/hwd-a53.dtb  binary 0x0b400000
restore ~/hwd/hwd10909_a53/linux/binge/ramdisk.img                        binary 0x15000000
```

相关最小需求 3 个模块

- kernel 二进制文件本身 Image(vmlinux 去掉符号表和调试信息后的二进制文件);
- kernel 设备树二进制文件;
- initrd 模块, 一般为 ramdisk 二进制文件;

1.3 kernel 引导

kernel 和相关配套模块加载到 ddr 后, 就可以在 u-boot 中引导其运行, 几种引导方式都大同小异.

1.3.1 booti 方式

booti 引导 Image, 引导需要 3 个模块地址 (booti [kernel_addr] [initrd_addr] [dtb_addr]);

```
# in u-boot
booti 0x8400000 0x10400000 0xb400000
```

1.3.2 bootz 方式

1.3.3 ram 文件系统

ram 文件系统是引导必不可少的组成部分. 有 initramfs 和 initrd 方式, 注意, 使 initramfs, 命令参数将不需要 "initrd=" 和 "root=" 命令.

- initramfs 概述: initramfs 与 initrd 类似, 也是初始化好了且存在于 ram 中的, 可以压缩也可以不压缩. 但是 前 initramfs 只 持 cpio 包格式, 它会被 populate_rootfs->unpack_to_rootfs(&__initramfs_start, &__initramfs_end - &__initramfs_start, 0) 函数 (解压缩、) 解析、安装。
- initramfs 与 initrd 区别

- Linux 内核只认 cpio 格式的 initramfs 件包 (因为 unpack_to_rootfs 只能解析 cpio 格式件), cpio 格式的 initramfs 件包将被系统抛弃, initrd 可以是 cpio 包也可以是传统的镜像 (image) 件, 实际使用中 initrd 都是传统镜像 件。
- initramfs 在编译内核的同时被编译并与内核连接成一个 件, 它被链接到地址 `__initramfs_start` 处, 与内核同时被 bootloader 加载到 ram 中, initrd 是另外单独编译成的, 是个独 的 件, 它由 bootloader 单独加载到 ram 中内核空间外的地址, 如加载的地址为 `addr`(是物理地址 虚拟地址), 为 8MB, 那么只要在命令 加 " `initrd=addr,8M`" 命令, 系统就可以找到 initrd(当然通过适当修改 Linux 的 录结构, makefile 件和相关代码, 以上两种情况都是可以相通的)。
- initramfs 被解析处理后原始的 cpio 包 (压缩或 压缩) 所占的空间 (`&__initramfs_start - &__initramfs_end`) 是作为系统的一部分直接保留在系统中, 不会被释放掉, 对于 initrd 镜像 件, 如果没有在命令 中设置 "keepinitrd" 命令, 那么 initrd 镜像 件被处理后其原始件所占的空间 (`initrd_end - initrd_start`) 将被释放掉。
- initramfs 可以独 ram disk 单独存在, 要 持 initrd 必须要先 持 ram disk, 即要配置 `CONFIG_BLK_DEV_INITRD` 选项- 持 initrd, 必须先要配置 `CONFIG_BLK_DEV_RAM`
- 持 ram disk, 因为 initrd image 实际就是初始化好了的 ramdisk 镜像 件, 最后都要解析、写 到 ram disk 设备 `/dev/ram` 或 `/dev/ram0` 中。

2 kernel 调试

kernel 调试可以使用 gdb 进行源码级调试, 如图1所示.

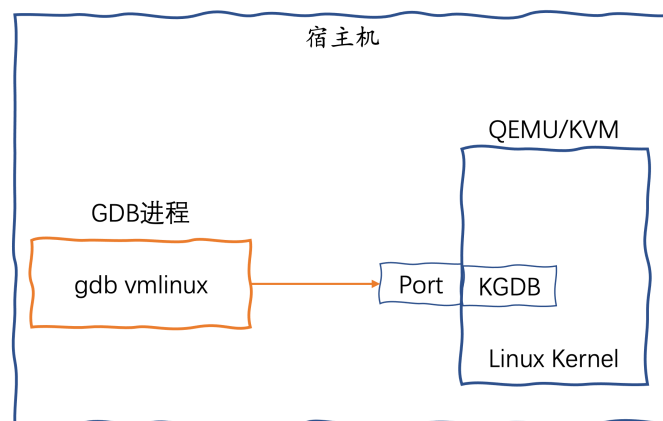


图 1: Kernel GDB 调试方案

- 使用主机的 gdb 作为 kernel 调试前端;
- 使用 KGDB 作为 kernel 调试的后端, 即 gdbserver 端.

为了构建能够调试的内核, 我们需要配置以下几个参数。

- `CONFIG_DEBUG_INFO`: 在内核和内核模块中包含调试信息, 这个选项在幕后为 gcc 使用的编译器参数增加了 -g 选项。这个选项的菜单路径如下;

```

Kernel hacking --->
Compile-time checks and compiler options --->
[*] Compile the kernel with debug info

```

- **CONFIG_FRAME_POINTER**: 这个选项会将调用帧信息保存在寄存器或堆栈上的不同位置, 使 **gdb** 在调试内核时可以更准确地构造堆栈回溯跟踪 (stack back traces);
- 启用 **CONFIG_GDB_SCRIPTS**, 但要关闭 **CONFIG_DEBUG_INFO_REDUCED**;
- **CONFIG_KGDB**: 启用内置的内核调试器, 该调试器允许进行远程调试;
- 关闭 **CONFIG_RANDOMIZE_BASE** 设置: **KASLR** 会更改引导时放置内核代码的基地址。如果你在 **内核配置** 中启用了 **KASLR** (**CONFIG_RANDOMIZE_BASE=y**), 则无法从 **gdb** 设置断点。

为了能让 **gdb** 在启动时能够加载 **Linux helper** 脚本, 需要在 **~/gdbinit** 文件中添加如下内容, **"/path/to/linux-build"** 就是 **Linux** 源码目录。

```
add-auto-load-safe-path /path/to/linux-build
```

在主机 **linux-source** 目录下执行如下命令。

```
gdb vmlinux
```

然后在 **gdb** 的交互环境下使用如下命令 **attach** 到 **linux** 内核。

```
target remote :1234
```

3 kernel 内存划分

在 **Kernel** 视角下, 内存划分如图2所示。

内存划分	low memory		high memory			
	DMA zone	normal zone	highmem zone			
内存分区	DMA内存动态分配	直接内存动态分配	隔离保护	高端内存动态分配	隔离保护	持久映射
地址空间划分	DMA内存动态分配	直接内存动态分配	隔离保护	高端内存动态分配	隔离保护	持久映射
起始地址	0	0	0	0	0	0
分配接口	kmalloc(GFP_ATOMIC)	kmalloc	gap(8MB)	vmalloc/vmap/ioremap	gap(2pages)	kmap
使用场景	kernel code/data			module/driver		fs/net

图 2: Kernel 内存视角

- **DMA 内存动态分配地址空间**: 一些 **DMA** 设备因为其自身寻址能力的限制, 不能访问所有内存空间。如早期的 **ISA** 设备只能在 24 位地址空间执行 **DMA**, 即只能访问前 16MB 内存。所以需要划分出 **DMA 内存动态分配空间**, 即 **DMA zone**。其分配通过加上 **GFP_ATOMIC** 控制符的 **kmalloc** 接口来申请。
- **直接内存动态分配地址空间**: 因为访问效率等原因, 内核对内存采用简单的线性映射, 但是因为 32 位 **CPU** 的寻址能力 (4G 大小) 和内核地址空间起始的设置 (3G 开始), 会导致内核的地址空间资源不足, 当内存大于 1GB 时, 就无法直接映射所有内存。无法直接映射的地址空间部分, 即 **highmem zone**。在 **DMA zone** 和 **highmem zone** 中间的区域即 **normal zone**, 主要用于内核的动态内存分配。其分配通过 **kmalloc** 接口来申请。
- **高端内存动态分配地址空间**: 高端内存分配的内存是虚拟地址连续而物理地址不连续的内存, 一般用于内核动态加载的模块和驱动, 因为内核可能运行了很久, 内存页面碎片情况严重, 如果要申请大的连续地址的内存页会比较困难, 容易导致分配失败。根据应用需要, 高端内存分配提供多个接口:
 - **vmalloc**: 指定分配大小, **page** 位置和虚拟地址隐式分配;
 - **vmap**: 指定 **page** 位置数组, 虚拟地址隐式分配;
 - **ioremap**: 指定物理地址和大小, 虚拟地址隐式分配。

- 持久映射地址空间: 内核上下文切换会伴随着 TLB 刷新, 这会导致性能下降。但一些使用高端内存的模块对性能也有很高要求。持久映射空间在内核上下文切换时, 其 TLB 不刷新, 所以它们映射的高端地址空间寻址效率较高。其分配通过 `kmap` 接口来申请。`kmap` 与 `vmap` 的区别是: `vmap` 可以映射一组 `page`, 即 `page` 不连续, 但虚拟地址连续, 而 `kmap` 只能映射一个 `page` 到虚拟地址空间。`kmap` 主要用于 `fs`、`net` 等对高端内存访问有较高性能要求的模块中。
- 固定映射地址空间: 持久映射的问题是可能会休眠, 在中断上下文、自旋锁临界区等不能阻塞的场景中不可用。为了解决这个问题, 内核又划分出固定映射, 其接口不会休眠。固定映射空间通过 `kmap_atomic` 接口来映射。`kmap_atomic` 的使用场景与 `kmap` 较为相似, 主要用于 `mm`、`fs`、`net` 等对高端内存访问有较高性能要求而且不能休眠的模块中。

SEGMENT (64bit)	SEGMENT (32bit)	BASE ADDR MACRO	BASE ADDR	FUNCTION	FLAG
kxseg	kseg2	CKSEG3	0xffffffff00000000	ko module	内核模式, MM映射, cache缓存
		CKXSEG	0xffffffff00000000	kernel code	内核模式, MM映射, cache缓存
	kseg1	NMI_BASE	0xffffffffb0000000	exception entry(reset, NMI etc.)	监督模式, MM映射, cache缓存
		DEFAULT_XLP_IO_BASE	0xffffffffb8000000	CPU iomem	内核模式, 直接映射, 非cache缓存
	kseg0	CKSEG1	0xffffffff00000000	pci bus iomem, i2c iomem	内核模式, 直接映射, 非cache缓存
		CKSEG0	0xffffffff00000000	tib entry, exception vector	内核模式, 直接映射, cache缓存
		PMAP_BASE	0xd000fe0000000000	HIGHMEM(unused)	
		VMALLOC_END	0xd000002000000000		
	只存在于64位CPU, 32位CPU不支持	VMALLOC_START	0xd000000000000000	vmalloc	内核模式, MM映射, cache缓存
		CKSEG	0xc000000000000000	kmalloc	
xxphys	不支持	XXPHYS_CAC_BASE	0x9800000000000000	kmap/kmap_atomic	内核模式, 直接映射, cache缓存
xsseg		XLP_XXPHYS_UNCACHED	0x9000000000000000	ioremap	内核模式, 直接映射, 非cache缓存
xsseg		XKXSEG	0x4000000000000000	supervisor space(unused)	监督模式, MM映射
xuseg	kuseg (2G)	XXUSEG	0x0000000000000000	user space	用户模式, MM映射

图 3: Kernel 内存视角细节

4 kernel 生成过程

- `vmlinux`: 依据 `arch/arm/kernel/vmlinux.lds` 生成 linux 内核源码根目录下的 `vmlinux`, 这个 `vmlinux` 属于未压缩, 带调试信息、符号表的最初的内核, 大小约 23MB;

```
xxx-ld -o vmlinux -T arch/arm/kernel/vmlinux.lds \
    arch/arm/kernel/head.o \
    init/built-in.o \
    --start-group \
    arch/arm/mach-s3c2410/built-in.o \
    kernel/built-in.o \
    mm/built-in.o \
    fs/built-in.o \
    ipc/built-in.o \
    drivers/built-in.o \
    net/built-in.o \
    --end-group \
    .tmp_kallsyms2.o
```

- `Image`: 将上面的 `vmlinux` 去除调试信息、注释、符号表等内容, 生成 `arch/arm/boot/Image`, 这是不带多余信息的 linux 内核, `Image` 的大小约 3.2MB;

```
xxx-objcopy -O binary -S vmlinux arch/arm/boot/Image
```

- `piggy.gz` 将 `arch/arm/boot/Image` 用 `gzip -9` 压缩生成 `arch/arm/boot/compressed/piggy.gz` 大小约 1.5MB;

```
gzip -f -9 < arch/arm/boot/Image > arch/arm/boot/compressed/piggy.gz
```

- 编译 arch/arm/boot/compressed/piggy.S 生成 arch/arm/boot/compressed/piggy.o 大小约 1.5MB, 这里实际上是将 piggy.gz 通过 piggy.S 编译进 piggy.o 文件中。而 piggy.S 文件仅有 6 行, 只是包含了文件 piggy.gz;

```
xxx-gcc -o arch/arm/boot/compressed/piggy.o \
arch/arm/boot/compressed/piggy.S
```

- 依据 arch/arm/boot/compressed/vmlinux.lds 将 arch/arm/boot/compressed/ 目录下的文件 head.o、piggy.o、misc.o 链接生成 arch/arm/boot/compressed/vmlinux, 这个 vmlinux 是经过压缩且含有自解压代码的内核, 大小约 1.5MB;

```
xxx-ld zreladdr=0x30008000 params_phys=0x30000100 \
-T arch/arm/boot/compressed/vmlinux.lds \
arch/arm/boot/compressed/head.o \
arch/arm/boot/compressed/piggy.o \
arch/arm/boot/compressed/misc.o \
-o arch/arm/boot/compressed/vmlinux
```

- 将 arch/arm/boot/compressed/vmlinux 去除调试信息、注释、符号表等内容, 生成 arch/arm/boot/zImage 大小约 1.5MB; 这已经是一个可以使用的 linux 内核映像文件了;

```
xxx-objcopy -O binary -S arch/arm/boot/compressed/vmlinux\
arch/arm/boot/zImage
```

- 将 arch/arm/boot/zImage 添加 64Bytes 的相关信息打包为 arch/arm/boot/uImage 大小约 1.5MB;

```
./mkimage -A arm -O linux -T kernel -C none \
-a 0x30008000 -e 0x30008000 \
-n 'Linux-2.6.35.7' \
-d arch/arm/boot/zImage arch/arm/boot/uImage
```

借用 biscuits 的一张图直观的描述下压缩内核生成过程:

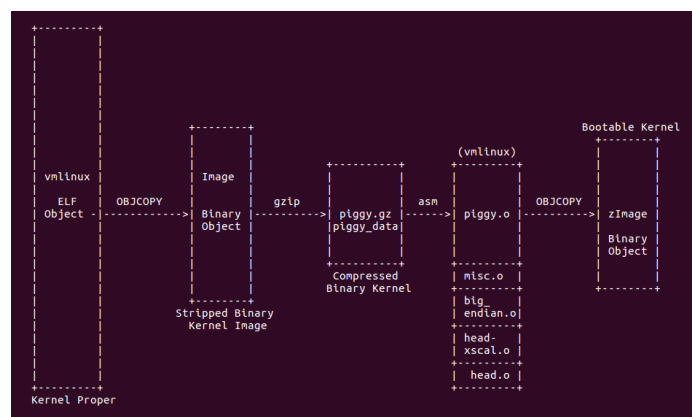


图 4: 内核编译过程

- uImage 格式: 压缩的 vmlinux+u-boot header;
- vmlinux: Linux kernel 的核心资产, elf 格式, 包含信息如下, 包含的主要部件见表1所示。
 - symbols;

- comments
- debug info
- architecture-generic components

表 1: vmlinux 主要部件

Component	Description
arch/arm/kernel/head.o	Kernel arch-specific startup code.
init_task.o	Initial thread & task structs.
init/built-in.o	Main kernel-initialization code.
usr/built-in.o	Built-in initramfs image.
arch/arm/kernel/built-in.o	Arch-specific kernel code.
arch/arm/mm/built-in.o	Arch-specific memory-management code.
arch/arm/common/built-in.o	Arch-specific generic code. Varies by arch.
arch/arm/mach-ixp4xx/built-in.o	Machine-specific code, usually initialization.
arch/arm/nwfpe/built-in.o	Arch-specific floating point-emulation code.
kernel/built-in.o	Common components of the kernel itself.
mm/built-in.o	Common components of memory-management code.
ipc/built-in.o	Interprocess communications, such as SysV IPC.
security/built-in.o	Linux security components.
lib/lib.a	Archive of miscellaneous helper functions.
arch/arm/lib/lib.a	Arch-specific common facilities.
lib/built-in.o	Common kernel helper functions.
drivers/built-in.o	All the built-in drivers not loadable modules.
sound/built-in.o	Sound drivers.
net/built-in.o	Linux networking.
.tmp_kallsyms2.o	Symbol table.

- System.map: 基于文本的内核 vmlinux 模块符号表;
- Image: 二进制的内核, 剥离了 symbols, notes, comments
- piggy.gz: 将二进制的 Image 进行 gzip 压缩后的结果。
- piggy.s: 包含对 piggy.gz 的引用指令。

```
.section .piggydata,#alloc
.globl input_data
input_data:
.incbin "arch/arm/boot/compressed/piggy.gz"
.globl input_data_end
input_data_end:
```

- zImage: 最终的内核镜像, 被 bootloader 引导;
- .version: 该文件用于记录编译序号, 每编译一次, scripts/mkversion 脚本会将里面的数字增加 1 次;
- 内核镜像构造过程见图5。

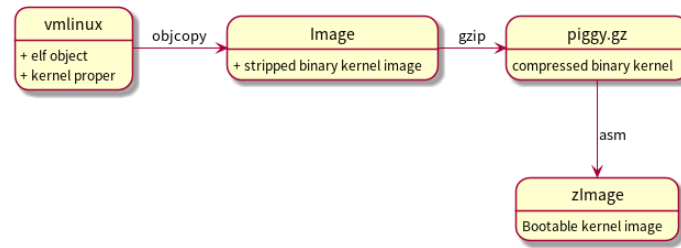


图 5: Composite Kernel Image Construction

对于 64 位的内核则和 32 位的有所不同. 具体参见图6所示.

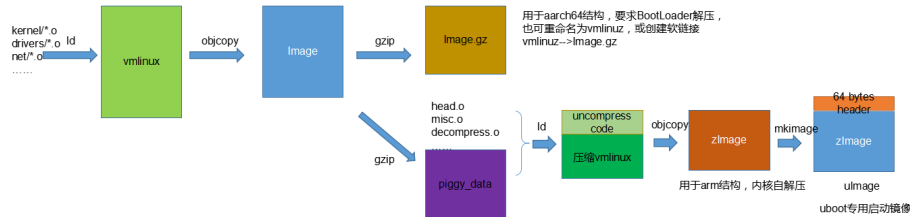


图 6: 64 位内核编译

5 代码分析

5.1 内核引导过程

Linux 内核启动是“BootROM->U-Boot-> 内核-> 用户空间”的控制权移交过程, 核心链路为“上电-> 固件自检->U-Boot 加载内核/initramfs-> 内核早期初始化->start_kernel-> 挂载根文件系统-> 启动 1 号进程-> 进入用户态”, 如图 8。

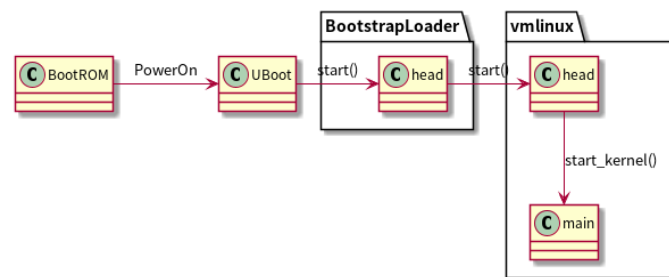


图 7: ARM Boot Control Flow

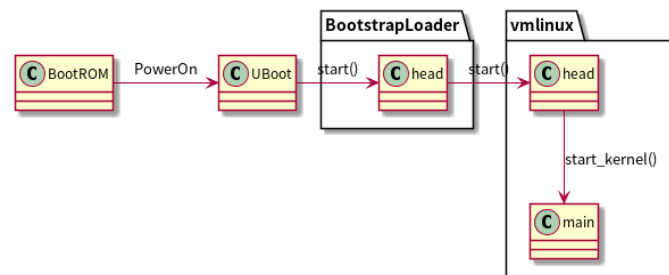


图 8: ARM Boot Control Flow

]]

5.2 zImage 入口函数

zImage 初始化阶段源码位于 arch/arm/boot/compressed/ 目录下。根据之前分析的原理可知, 压缩之后的内核会添加 bootstrap 功能之后生成 vmlinux, 再经过 OBJCOPY 工具处理生成 zImage。所以可以通过查看 vmlinux 的链接脚本确定 zImage 的入口地址。zImage 使用的链接脚本位于 arch/arm/boot/compressed/vmlinux.lds.S, 具体内容如下

```
OUTPUT_ARCH(arm)
ENTRY(_start)
SECTIONS
{
    /DISCARD/ : {
        *(.ARM.exidx*)
        *(.ARM.extab*)
        /*
         * Discard any r/w data - this produces a link error if we have any,
         * which is required for PIC decompression. Local data generates
         * GOTOFF relocations, which prevents it being relocated independently
         * of the text/got segments.
         */
        *(.data)
    }

    . = TEXT_START;
    _text = .;

    .text : {
        _start = .;
        *(.start)
        *(.text)
        *(.text.*)
        *(.fixup)
        *(.gnu.warning)
        *(.glue_7t)
        *(.glue_7)
    }
}
```

从链接脚本可以知道 vmlinux 链接过程, 使用 ENTRY 关键字指定了 vmlinux 的入口地址, 也就是第一行运行的代码, 这里设置为 _start, 从上面可以看出 _start 位于 .text section 的首地址, 所以这里链接脚本告诉开发者, vmlinux 运行的第一行代码就是 vmlinux .text section 的第一行代码。继续查看链接脚本, .text section 的布局是所有目标文件的 .start section 位于 vmlinux .text section 的最前部, 所以开发者只需找到目标文件中函数 .start section 的文件即可。由 arch/arm/boot/compressed/Makefile 可知, head.o 第一个被链接进 vmlinx, start 段为 heads 中的 start section

```
HEAD      = head.o
$(obj)/vmlinux: $(obj)/vmlinux.lds $(obj)/$(HEAD) $(obj)/piggy.o \
    $(addprefix $(obj)/, $(OBJS)) $(lib1funcs) $(ashldi3) \
    $(bswapsdi2) $(efi-obj-y) FORCE
    @$(check_for_multiple_zreladdr)
    $(call if_changed,ld)
    @$(check_for_bad_syms)
```

找到 start 开始的代码如下

```
.section ".start", #alloc, #execinstr      //属性是可分配和可执行的
/**
 * sort out different calling conventions
 */
.align
/**
 * Always enter in ARM state for CPUs that support the ARM ISA.
 * As of today (2014) that's exactly the members of the A and R
 * classes.
 */
AR_CLASS(.arm) //.align 伪指令和 .arm 伪指令告诉汇编器，这是一个使用 arm32 指令集并 要求对齐的 se
start:
    .type      start,#function
    .rept      7 @重复空操作7次
    __nop
    .endr
    ARM(mov    r0, r0)
    ARM(b      1f) @跳转到1处
    THUMB(badr r12, 1f)
    THUMB(bx    r12)

    .word      _magic_sig      @ Magic numbers to help the loader
                                @ 魔数0x016f2818是在uboot中用于判断zImage的存在
                                @ 这个是内核和uboot约定好的。
    .word      _magic_start @ absolute load/run zImage address 用于指定zImage的加载和运行的绝对
    .word      _magic_end    @ zImage end address
    .word      0x04030201    @ endianness flag 表示字节序标志

    THUMB(.thumb)
1:      __EFI_HEADER //指向 EFI 头

    ARM_BE8(setend    be)      @ go BE8 if compiled for BE8
    AR_CLASS(mrs      r9, cpsr)
#ifdef CONFIG_ARM_VIRT_EXT
    bl __hyp_stub_install @ get into SVC mode, reversibly
#endif

    @ r1和r2中分别存放着由uboot传递过来的
    @ architecture ID和指向标记列表的指针。
    mov r7, r1 @ save architecture ID
    mov r8, r2 @ save atags pointer

#ifdef CONFIG_CPU_V7M
    /**
     * Booting from Angel - need to enter SVC mode and disable
     * FIQs/IRQs (numeric definitions from angel arm.h source).
     * We only do this if we were in user mode on entry.
     */
```

```

        mrs    r2, cpsr    @get current mode
        tst    r2, #3      @not user?
        bne    not_angel
        mov    r0, #0x17   @angel_SWIreason_EnterSVC
ARM(swi 0x123456) @ angel_SWI_ARM  angel_SWI_ARM 0x123456是arm指令集的半主机操作编号
THUMB(svc 0xab)   @ angel_SWI_THUMB
not_angel:
        safe_svcmode_maskall r0 @进入svc模式
        msr    spsr_cxsf, r9    @ Save the CPU boot mode in
                                @ SPSR

#endif

```

接下来, head.S 将找到物理地址的起始地址, 这个时候 MMU 是没有打开的, 这个时候是忽略任何地址对齐和偏移。head.S 选择最开始的 128MB 处作为对齐地址, 然后将 zImage 放在这物理地址起始处, 这 128MB 就是用来专门存放 zImage 镜像的。具体代码如下:

```

#ifdef CONFIG_AUTO_ZRELADDR
/**
 * Find the start of physical memory.  As we are executing
 * without the MMU on, we are in the physical address space.
 * We just need to get rid of any offset by aligning the
 * address.
 *
 * This alignment is a balance between the requirements of
 * different platforms - we have chosen 128MB to allow
 * platforms which align the start of their physical memory
 * to 128MB to use this feature, while allowing the zImage
 * to be placed within the first 128MB of memory on other
 * platforms.  Increasing the alignment means we place
 * stricter alignment requirements on the start of physical
 * memory, but relaxing it means that we break people who
 * are already placing their zImage in (eg) the top 64MB
 * of this range.
 */
        mov    r4, pc
        and    r4, r4, #0xf8000000
        /* Determine final kernel image address. */
        add    r4, r4, #TEXT_OFFSET
#else
        ldr    r4, =zreladdr
#endif

```

这段代码主要用于计算内核的解压地址, 并将解压地址存储到 r4 寄存器中。首先调用代码“mov, r4, pc”, 将当前 CPU 执行的地址存储到 r4, 然后在将 0xf8000000 的值与 r4 相与达到对齐的作用, 确保之后的内核解压地址按 128M 对齐, 然后将 r4 寄存器的值加上 TEXT_OFFSET, TEXT_OFFSET 代表内核的解压地址, 这样 r4 寄存器就存储了内核的解压地址。TEXT_OFFSET 定义在 arch/arm/Makefile 中, 如下:

```

# Text offset. This list is sorted numerically by address in order to
# provide a means to avoid/resolve conflicts in multi-arch kernels.

```

```

textofs-y      := 0x00008000
# We don't want the htc bootloader to corrupt kernel during resume
textofs-$(CONFIG_PM_H1940)      := 0x00108000
# SA1111 DMA bug: we don't want the kernel to live in precious DMA-able memory
ifeq ($(CONFIG_ARCH_SA1100),y)
    textofs-$(CONFIG_SA1111) := 0x00208000
endif

    textofs-$(CONFIG_ARCH_MSM8X60) := 0x00208000
    textofs-$(CONFIG_ARCH_MSM8960) := 0x00208000
    textofs-$(CONFIG_ARCH_AXxia) := 0x00308000
# The byte offset of the kernel image in RAM from the start of RAM.TEXT_OFFSET := $
(textofs-y)

```

这里根据自己环境的芯片类型选择 offset(:= 表示覆盖之前的赋值)。这里分析按照 0x00008000 进行计算。接着执行如下代码

```

/**
 * Set up a page table only if it won't overwrite ourself.
 * That means r4 < pc || r4 - 16k page directory > &_end.
 * Given that r4 > &_end is most unfrequent, we add a rough
 * additional 1MB of room for a possible appended DTB.
 */
mov    r0, pc
cmp    r0, r4 //此操作影响状态寄存器中的C状态位.
ldrcc  r0, LC0+32
addcc  r0, r0, pc
cmpcc  r4, r0
orrcc  r4, r4, #1          @ remember we skipped cache_on
blcs   cache_on //如果pc>r4则直接执行这里

```

其中 LC0 定义如下

```

.align    2
.type     LC0, #object
LC0:
    .word  LC0          @r1
    .word  __bss_start   @r2
    .word  _end          @r3
    .word  _edata        @r6
    .word  input_data_end - 4    @ r10 (inflated size location)
    .word  _got_start     @r11
    .word  _got_end       @ip
    .word  .L_user_stack_end @ sp
//LC0+32位置, zImage 需要重定位的长度再加上 “16K + 1M” 的长度
    .word  _end - restart + 16384 + 1024*1024
    .size  LC0, . - LC0

```

这段代码的主要任务就是确认 zImage 自己建立的页表会不会被 zImage 镜像的重定位给覆盖掉。从原理可以知道, zImage 被加载到内存运行之后, 会将自己重定位到新的物理地址运行, 这就会出现要创建的页表可能被 zImage 重定位之后覆盖。zImage 镜像如果不被自己给覆盖, 需要满足两个条件中的任意一个:

pc当前地址: f8000000+0x**
顺序执行解压从这里开始, 不会覆盖在执行的pc地址的数据
r4: 128M对齐+offset后地址

这种情况下, 内核的解压地址小于当前 PC 运行物理地址。

$$r4 - 16k_{pagedirectory} >_{end}$$

这种情况下, 内核的解压地址大于 zImage 结束地址之后的 16KB。一般情况下解压内核的地址大于 zImage 的结束地址是不太寻常的。这种情况下需要添加 1MB 的空间与链接在 zImage 中的 DTB 隔开。

运行到这里, 首先获得 PC 寄存器的值, 然后调用“cmp r0, r4”, 从之前的代码可知, r4 寄存器存储着解压内核的地址, 这里执行这条命令的含义是, 如果 $r0 > r4$, 那么代表当前 PC 执行地址大于内核解压地址, 这种情况符合之前的讨论, 所以那么就执行 cache_on 宏; 如果 $r0 < r4$, 那么 zImage 的运行范围包含了要解压内核的地址, 因此需要继续进行检测。这里执行命令“ldrcc r0, LC0+32”, 通过这个命令, 将 LC0 偏移 32 个字节地址对应的内容拷贝到 r0 寄存器。这里再加上 PC 的值, 确保当前运行的代码也不会被覆盖。接着执行命令“cmpcc r4, r0”, 重新确定解压内核的物理地址与 zImage 重定位的物理地址是否存在重叠。如果存在, 那么将 r4 寄存器的值加 1, 以此标记 cache_on 被跳过; 否则执行“blcs cache_on”打开缓存。cache_on 代码如下

```
/**
 * Turn on the cache. We need to setup some page tables so that we
 * can have both the I and D caches on.
 *
 * We place the page tables 16k down from the kernel execution address,
 * and we hope that nothing else is using it. If we're using it, we
 * will go pop!
 *
 * On entry,
```

```

* r4 = kernel execution address
* r7 = architecture number
* r8 = atags pointer
* On exit,
* r0, r1, r2, r3, r9, r10, r12 corrupted
* This routine must preserve:
* r4, r7, r8
*/
    .align    5
cache_on:    mov    r3, #8 @ cache_on function
            b      call_cache_fn

```

cache_on 用于打开 ARM 的 cache 功能, cache 包括 I-Cache (指令 cache) 和 D-cache (数据 cache)。为了正常使用 cache, 需要建立一个页表供 MMU 使用。zImage 会将这个页表放在真正内核开始执行地址之前的 16K 位置, 希望这个地址不要被非法使用。在执行 cache_on 之前, r4 寄存器里存储值内核解压地址, 这个地址也被成为真正内核开始执行的地址。r7 寄存器存储这体系相关的数据; r8 寄存器存储着 uboot 传递给内核 ATAG 参数的地址。cache_on 使用伪指令 .align 指定了对齐方式为 5 字节对齐。由于 ARM 将所有家族芯片关于 cache 的操作都放在一个表里维护, 因此 armv7 的 cache 操作也在这个表内, 并且 cache_on 操作在表中的偏移是 8, 因此代码将立即数 8 存储到 r3 寄存器, 并跳转到 call_cache_fn 出执行, 代码如下:

```

call_cache_fn:    adr    r12, proc_types
#ifdef CONFIG_CPU_CP15
    mrc    p15, 0, r9, c0, c0    @ get processor ID
#elif defined(CONFIG_CPU_V7M)
    /*
    * On v7-M the processor id is located in the V7M_SCB_CPUID
    * register, but as cache handling is IMPLEMENTATION DEFINED on
    * v7-M (if existant at all) we just return early here.
    * If V7M_SCB_CPUID were used the cpu ID functions (i.e.
    * __armv7_mmu_cache_{on,off,flush}) would be selected which
    * use cp15 registers that are not implemented on v7-M.
    */
    bx     lr
#else
    ldr    r9, =CONFIG_PROCESSOR_ID
#endif
1:
    ldr    r1, [r12, #0]          @ get value
    ldr    r2, [r12, #4]          @ get mask
    eor    r1, r1, r9             @ (real ^ match)
    tst    r1, r2                 @      & mask
    ARM(addeq pc, r12, r3) @ call cache function
    THUMB(addeq    r12, r3)
    THUMB(moveq    pc, r12) @ call cache function
    add    r12, r12, #PROC_ENTRY_SIZE
    b      1b

```

本段代码的主要任务就是 cache_on 表中找到 armv7 对应的 cache_on 操作。代码首先使用 adr 伪指令获得 proc_types 的地址, 然后在 CONFIG_CPU_CP15 宏启用的情况下, armv7 这个宏启用, 使用

mrc 指令,读取了 CPU ID 寄存器。

将 ID 相关的信息存储在 r9 寄存器里,然后使用 ldr 指令,从 proc_types 表里按顺序取出每个成员的第一个值和第二个值,分别存储在 r1, r2 寄存器中。接着代码将 r1 寄存器的值与 r9 寄存器的值做异或运算,然后将结果存储到 r1 寄存器,接着再调用 tst 命令,将 r1 寄存器与 r2 寄存器按位与操作。如果结果为零,那么就执行 addeq 指令,将 pc 指向表中的位置;如果结果不为零,那么调用 add 指令,将 r12 指向下一个表成员,调用 b 指令继续循环。

表 proc_types 定义为一个 object 对象,其中包含了 armv7 对应的成员,cache_on 对应的入口函数就是: __armv7_mmu_cache_on,结构如下:

```
/**
 * Table for cache operations. This is basically:
 * - CPU ID match
 * - CPU ID mask
 * - 'cache on' method instruction
 * - 'cache off' method instruction
 * - 'cache flush' method instruction
 *
 * We match an entry using: ((real_id ^ match) & mask) == 0
 *
 * Writethrough caches generally only need 'on' and 'off'
 * methods. Writeback caches _must_ have the flush method
 * defined.
 */
.align 2
.type proc_types, #object
proc_types:
    .word 0x41000000 @ old ARM ID
    .word 0xff00f000
    mov pc, lr
    THUMB(nop)
    mov pc, lr
    THUMB(nop)
    mov pc, lr
    THUMB(nop)
    .word 0x000f0000 @ new CPU Id
    .word 0x000f0000
    W(b) __armv7_mmu_cache_on
    W(b) __armv7_mmu_cache_off
    W(b) __armv7_mmu_cache_flush
```

__armv7_mmu_cache_on 函数首先将 lr 寄存器的值存储到 r12 寄存器,然后判断 CONFIG_MMU 宏是否启用。在 armv7 里,这个宏是启用的,所以执行宏限定的代码。首先也是通过 mrc 指令读取 ID_MMFR0 寄存器的值到 r11 寄存器中,其中, ID_MMFR0 的最低位用于指示当前 CPU 是支持 VMSA 还是 PMSA。VMSA 指的是 Virtual Memory System Architecture。PMSA 指的是 Protected Memory System Architecture。两种模式具有不同的内存管理策略,具体介绍开发者可以参考 ARMv7 Reference Manual。接着使用 tst 指令查看 r11 寄存器的最低 4 位的值,以此判断目前 CPU 是 VMSA 模式还是 PMSA 模式。如果 r11 寄存器的最低 4 bit 不为零,即 CPU 支持 VMSA 模式,那么代码继续执行带 ne 条件的指令。

__armv7_mmu_cache_on:

```

    mov r12, lr
#ifdef CONFIG_MMU
    mrc p15, 0, r11, c0, c1, 4    @ read ID_MMFR0
    tst r11, #0xf @ VMSA
    movne r6, #CB_BITS | 0x02    @ !XN
    blne __setup_mmu

```

CPU 支持 VMSA, 将 CB_BITS 或上 0x2 的值存储到 r6 寄存器, 并将跳转到 __setup_mmu 继续执行

```

__setup_mmu:
    sub    r3, r4, #16384 @ Page directory size
    bic    r3, r3, #0xff  @ Align the pointer
    bic    r3, r3, #0x3f00

```

__setup_mmu 的主要功能是建立一个临时页表, 并打开 MMU, 以此供解压程序使用虚拟地址。正如上面的代码所示, 此时 r4 寄存器指向真正内核运行的起始地址, 那么就在这个地址前 16K 处开始建立页表, 将页表的起始地址存储到 r3 寄存器, 并使用 bic 指令对 r3 的地址做对齐。所以页表和内核的位置关系如下图所示:



继续执行 __setup_mmu 中如下代码:

```

/**
 * Initialise the page tables, turning on the cacheable and bufferable
 * bits for the RAM area only.
 */
mov    r0, r3
mov    r9, r0, lsr #18
mov    r9, r9, lsl #18    @ start of RAM
add    r10, r9, #0x10000000 @ a reasonable RAM size
mov    r1, #0x12          @ XN|U + section mapping
orr    r1, r1, #3 << 10   @ AP=11
add    r2, r3, #16384

```

代码首先将 r3 寄存器的值存储到 r0 寄存器, 并通过对 r0 寄存器按 (1<<18) 对齐, 获得 RAM 的起始地址, 然后假设 RAM 的长度大概是 256M, 并将 RAM 结束地址存放在 r10 寄存器中。这里这样做的目的是: 该阶段, 内核采用一个临时页表, 页表按 1:1 映射物理地址与虚拟地址, 通过计算获得 RAM 的长度, 以此对能真实映射 RAM 的页表项设置一种标志集; 同理对不能映射物理地址的页表项设置另外一种标志集。接着将 0x12 的值和 (3<<10) 值存储到 r1 寄存器中。将 r3 寄存器的值加上 16K 存放到 r2 寄存器, 主要是为了防止写页表时越界。上面的左移 18 bit, 再右移 18 bit, 主要是按 1M 页表进行对齐。继续执行如下代码:

1:

```

cmp    r1, r9          @ if virt > start of RAM
cmphs  r10, r1         @  && end of RAM > virt
bic    r1, r1, #0x1c    @ clear XN|U + C + B
orrlo  r1, r1, #0x10    @ Set XN|U for non-RAM
orrhs  r1, r1, r6       @ set RAM section settings
str    r1, [r0], #4     @ 1:1 mapping
add    r1, r1, #1048576
teq    r0, r2
bne    1b

```

这段代码的主要任务就是设置各个页表项。有前面的代码可以知道, r1 存储着虚拟地址, 并且从虚拟地址 0 开始。r9 寄存器值存储着物理起始地址。上面代码的逻辑基本可以归纳为当虚拟地址不在 RAM 对于的物理地址上, 那么执行 bic 指令将 r1 寄存器的 0x1c 对应的位清理, 然后将 r1 与 0x10 做或运算, 以此标记这类页表; 如果虚拟地址在 RAM 对于的物理地址上, 那么执行 bic 指令将 r1 寄存器的 0x1c 对应的位清理, 然后将 r6 对应的标志与 r1 相与。经过上面的处理之后调用 str 指令将 r1 的值写入 r0 寄存器对应的内存里, 然后将 r0 寄存器的值加上 4。然后将 r1 寄存器的值加上 1M, 如果 r0 的值小于 r2, 那么跳转到 1 处继续循环写页表。通过上面分析, 可以获得两种页表的表示分别是: 1) 0xc12 (不映射 RAM) 2) 0xc0e (映射 RAM)。

/**

```

* If ever we are running from Flash, then we surely want the cache
* to be enabled also for our execution instance... We map 2MB of it
* so there is no map overlap problem for up to 1 MB compressed kernel.
* If the execution is in RAM then we would only be duplicating the above.
*/

```

```

orr    r1, r6, #0x04 @ ensure B is set for this
orr    r1, r1, #3 << 10
mov    r2, pc
mov    r2, r2, lsr #20
orr    r1, r1, r2, lsl #20
add    r0, r3, r2, lsl #2
str    r1, [r0], #4
add    r1, r1, #1048576
str    r1, [r0]
mov    pc, lr

```

ENDPROC(__setup_mmu)

接下来这段代码主要的目的就是为了区分系统是从 Flash 启动, 如果是, 那么就将 RAM 对应的页表的前 2M 设置特殊的页表标志。此时经过映射之后, r1 指向了虚拟地址 0, 然后调用 orr 指令将 r1 寄存器存储特定的标志。通过将 PC 的值传入 r2 寄存器, 并计算出 r2 对应的页表中的偏移, 然后写入 r1 中的值到页表中, 然后在将 r1 的值指向下 1M 地址空间, 最后将 lr 的值传递给 pc 寄存器, 那么函数至此返回。接着返回 __setup_mmu 的调用点。执行如下代码:

__armv7_mmu_cache_on:

```

mov    r12, lr

```

#ifdef CONFIG_MMU

```

mrc    p15, 0, r11, c0, c1, 4 @ read ID_MMFR0
tst    r11, #0xf                @ VMSA
movne  r6, #CB_BITS | 0x2      @ !XN

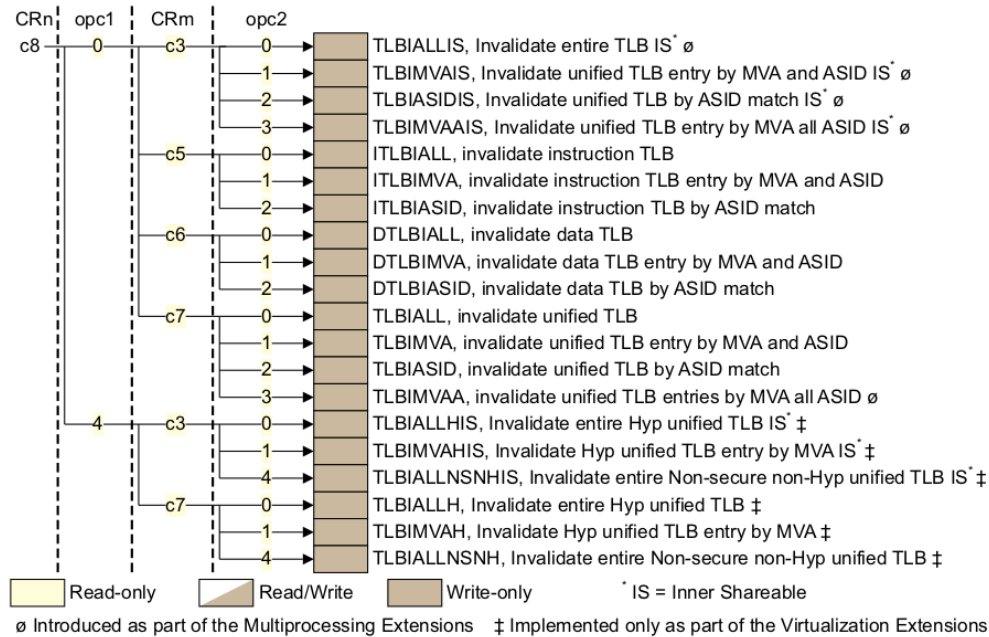
```

```

blne    __setup_mmu
mov     r0, #0
mcr     p15, 0, r0, c7, c10, 4 @ drain write buffer
tst     r11, #0xf                @ VMSA
mcrne   p15, 0, r0, c8, c7, 0   @ flush I,D TLBs
#endif

```

从 __setup_mmu 返回之后，代码首先将 r0 寄存器设置为 0，然后调用 mcr 指令实现以此 DMB，也就是内存屏障，保证这条指令之前所有内存访问都必须完成。继续使用 tst 指令确定当前模式是 VMSA 模式。如果是 VMSA 模式，那么就是调用 mcrne 指令，此时 CP15 调用情况如下图：



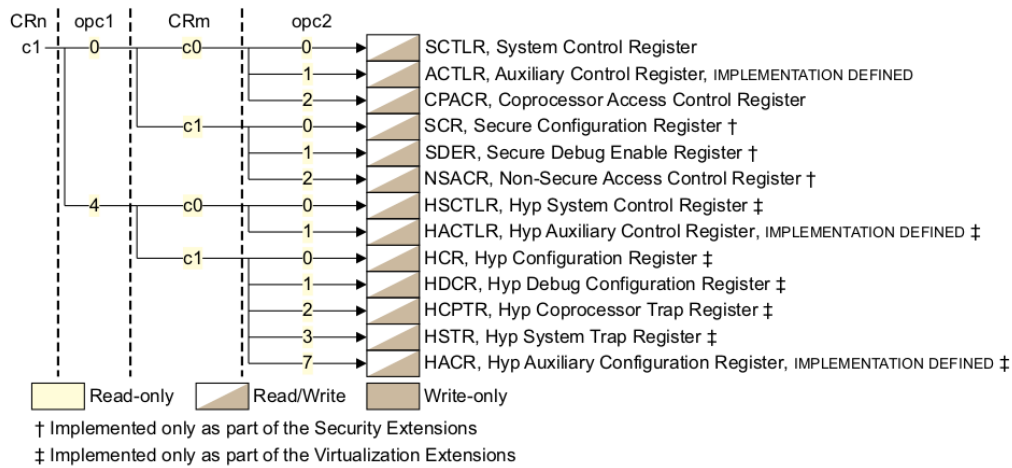
因此此时选择的是 TLBIALl 寄存器，向该寄存器写入任何值都会影响刷 I-TLB 和 D-TLB。这在 MMU 启用之前是必要的。接下来执行的代码是：

```

mrc     p15, 0, r0, c1, c0, 0 @ read control reg
bic     r0, r0, #1 << 28      @ clear SCTLr.TRE
orr     r0, r0, #0x5000       @ I-cache enable, RR cache replacement
orr     r0, r0, #0x003c       @ write buffer
bic     r0, r0, #2            @ A (no unaligned access fault)
orr     r0, r0, #1 << 22      @ U (v6 unaligned access model)
                                @ (needed for ARM1176)

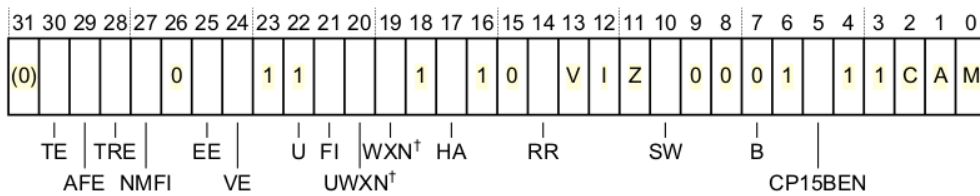
```

首先通过 mrc 指令读取 CP15 寄存器，该寄存器的布局如下：



选择 SCTL 寄存器, 其位布局图如下:

In a VMSAv7 implementation, the SCTL bit assignments are:

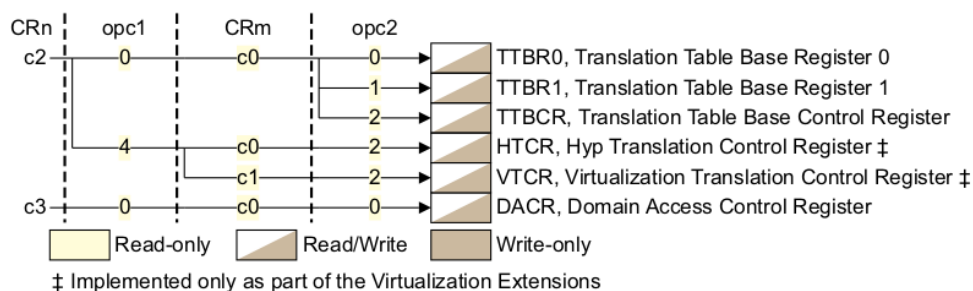


† Reserved before the introduction of the Virtualization Extensions, see text for more information.

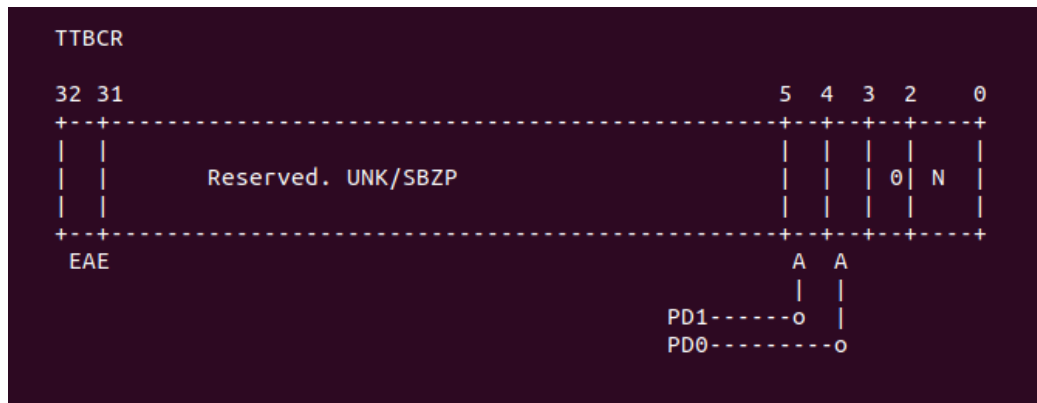
接着就是对 SCTL 寄存器特定位的操作, 首先清楚掉 TRE 位, 然后选中 I-cache enable, RR cache 替代算法, 写缓存, 设置对齐方式, 具体细节可以查看 ARMv7 Reference Manual。接下来执行的代码如下:

```
#ifdef CONFIG_MMU
    mrcne    p15, 0, r6, c2, c0, 2    @ read ttb control reg
    orrne    r0, r0, #1                @ MMU enable
    movne    r1, #0xffffffffd         @ domain 0 = client
    bic      r6, r6, #1 << 31         @ 32-bit translation system
    bic      r6, r6, #(7 << 0) | (1 << 4) @ use only ttbr0
    mcrne    p15, 0, r3, c2, c0, 0    @ load page table pointer
    mcrne    p15, 0, r1, c3, c0, 0    @ load domain access control
    mcrne    p15, 0, r6, c2, c0, 2    @ load ttb control
#endif
```

接下来的代码是设置 MMU 最重要的代码, 这段代码主要任务就是设置页表的基地址寄存器, 并将该寄存器的值指向了页表的基地址。首先调用 mrcne 指令, 代表了在 VMSA 模式下, 读取 TTB 控制器, 对应的 CP15 c2 寄存器如下:



选择了 TTBCR 寄存器, 对应的位图如下:



继续上面 r0 寄存器, 设置了 r0 最地位, 这里对应这 MMU enable 位, 置位之后一旦写入 SCTR 寄存器, 那么 MMU 就可以使用了。domain 的访问设置为 client。对于 TTBCR 寄存器, 将 31 bit 清理, 以此支持 32 位的地址转换, 然后对 TTBCR 寄存器, 清楚低 3 位和第 4 位, 以此告诉页表只是用 TTBR0 作为页表的基地址。接下来是将值写到 CP15 对应的寄存器上, 第一条指令是“mcrne p15, 0, r3, c2, c0, 0”, 这条命令的作用是将页表的基地址存储到 TTBR0 寄存器, 从上面的代码可以知道, r3 寄存器一直存储着页表的基地址。第二条指令是“mcrne p15, 0, r1, c3, c0, 0”, 设置了 domain 的访问域。第三条指令是“mcrne p15, 0, r6, c2, c0, 2”, 告诉页表控制器, 目前使用的页表是 32 位装换方式, 并且只使用 TTBR0 寄存器作为页表的基地址。接下来执行的代码是:

```
mcr    p15, 0, r0, c7, c5, 4    @ ISB
mcr    p15, 0, r0, c1, c0, 0    @ load control register
mrc    p15, 0, r0, c1, c0, 0    @ and read it back
mov    r0, #0
mcr    p15, 0, r0, c7, c5, 4    @ ISB
mov    pc, r12
```

运行到最后阶段, 将这前设置好的值写入到对应的寄存器中。首先执行以此 ISB 指令, 将流水线, 内存访问操作全部 flush 一次。接着执行命令“mcr p15, 0, r0, c1, c0, 0”, 将之前关于 MMU enable/I-cache/D-cache 等在 SCTR 控制器的配置全部写入到 SCTR 寄存器中, 写入之后, 系统立即生效。至此 MMU 和 I-cache 和 D-cache 都能使用。这里再次调用 ISB 指令, 将流水线上的指令等同步到最新的配置。最后将 r12 的返回地址赋值给 pc, 实现函数返回。

在启用 MMU 之后, 此时物理地址和虚拟地址按 1:1 映射。回到之前执行代码的位置继续执行如下代码:

```
restart:
    adr    r0, LC0
    ldmia  r0, {r1, r2, r3, r6, r10, r11, r12}
    ldr    sp, [r0, #28]
```

这段代码很简单, 就是 zImage 阶段创建了一个简单的表 LC0, 然后将 LC0 表的内容分别赋值给指定的寄存器。LC0 表用于存储 zImage 链接阶段各个重要段的偏移值, 这里首先查看一下 LC0 表的内容:

```
.align 2
.type   LC0, #object
LC0:
.word   LC0                @ r1
.word   __bss_start        @ r2
.word   _end               @ r3
.word   _edata             @ r6
```

```

.word    input_data_end - 4      @ r10 (inflated size location)
.word    _got_start              @ r11
.word    _got_end                @ ip
.word    .L_user_stack_end      @ sp
.word    _end - restart + 16384 + 1024*1024
.size    LC0, . - LC0

```

从上面的定义可以知道:

- LC0 + 0: LC0 在 zImage 中的偏移地址
- LC0 + 4: BSS 段起始地址在 zImage 镜像中的偏移地址
- LC0 + 8: BSS 段结束地址在 zImage 镜像中的偏移地址
- LC0 + 12: 压缩内核的长度在 zImage 镜像中的偏移地址
- LC0 + 16: zImage GOT 表起始地址在 zImage 镜像中的偏移地址
- LC0 + 20: zImage GOT 表结束的地址 zImage 镜像中的偏移地址
- LC0 + 24: zImage 堆栈的位置
- LC0 + 28: zImage 重定位的长度
- LC0 + 32: LC0 表长

```

/**
 * We might be running at a different address. We need
 * to fix up various pointers.
 */
sub      r0, r0, r1      @ cacalculate the delta offset
add      r6, r6, r0      @ _edata
add      r10, r10, r0    @ inflated kernel size location

```

由于 LC0 表内的值有的是链接时相对于 zImage 镜像的偏移值,此时开发者需要通过这些值加载到内存之后对应的值,因此上面的代码就是用于矫正 LC0 内的新值。从之前的代码可以知道, r0 寄存器存储的值是 LC0 表在内存中的地址,此时由于 MMU 已经启用,因此 r0 的值代表 LC0 表的虚拟地址, r1 寄存器存储着 LC0 相对于 zImage 镜像的偏移值。因此命令“sub r0, r0, r1”可以计算 LC0 表的基础偏移,然后表内各项的偏移都可以通过这个值进行计算。

r10 寄存器经过矫正之后对应这内核原始大小,那么原始内核大小是如何被存放到 LC0 表里呢?回答这个问题首先应该明确几点:解压后的内核就是之前所说的 Image, Image 有完整的 vmlinux 经过 OBJCOPY 命令生成的二进制文件,这个 Image 是可以直接在内存上运行的,所以知道 Image 长度是一个至关重要的问题。那么下面介绍一下编译系统 Kbuild 是如何计算 Image 长度呢?

首先 Image 经过压缩之后获得压缩内核 piggy_data,其使用的压缩命令如下:

```

$(obj)/piggy_data: $(obj)/../Image FORCE
    $(call if_changed,$(compress-y))

```

这段代码位于 arch/arm/boot/compressed/Makefile,这段代码就是 Image 压缩生成 piggy_data 压缩内核的过程,具体使用哪种压缩方法,通过 compress-y 决定,其定义也在同一个文件中,如下:

```

compress-$(CONFIG_KERNEL_GZIP) = gzip
compress-$(CONFIG_KERNEL_LZO)  = lzo
compress-$(CONFIG_KERNEL_LZMA) = lzma
compress-$(CONFIG_KERNEL_XZ)   = xzkernel
compress-$(CONFIG_KERNEL_LZ4)  = lz4

```

从上面的定义可知,内核支持多种压缩方式,其中以 gzip 为例,piggy_data 的压缩命令是:

```
$(obj)/piggy_data: $(obj)/../Image FORCE
    $(call if_changed,gzip)
```

Kbuild 的命令库里查看这个命令的具体过程,Kbuild 的命令库位于源码 scripts/Makefile.lib, gzip 命令如下:

```
quiet_cmd_gzip = GZIP      $@
    cmd_gzip = cat $(filter-out FORCE,$^) | gzip -n -f -9 > $@
```

所以可以看到 gzip 的执行过程,这里有个很重要的概念:压缩工具无论进行何种压缩算法,会在压缩文件的最后四个字节存储原始文件的大小,并按大端的模式存储。例如使用工具分别查看 Image 的大小以及 piggy_data 的最后四个字节,如下:

```
$ ll arch/arm/boot/Image
-rwxrwxr-x 1 buddy buddy 11931944 4月  1 07:06 Image*

$ bless arch/arm/boot/compressed/piggy_data
EB 11 97 AB C6 BC B8 40 5D 87 38 5B 35 E6 05 45 6A EC 99 66 4F 5F
49 A3 3F 96 EB A4 1D 2B E9 1A BB 1B D7 B7 78 F5 80 26 4E 4E FF 0F
A1 10 18 DA 28 11 B6 00
```

通过上面的数据分析可到, piggy_data 的最后四个字节是 28 11 B6 00,按小端调整之后的值是 0x00B61128,对应十进制值是 11931944,数值正好对应 Image 的长度,因此上面的推论是正确的。这是 piggy_data 的最后四字节存储着 Image 的长度,根据原理可以知道 Kbuild 编译系统将 piggy.S 汇编文件将 piggy_data 二进制文件封装成一个汇编文件,并链接成一个 ELF 文件,并在 piggy.S 中定义了两个全局符号:“input_data”和“input_data_end”,这两个符号标记了 piggy.o 里 piggy_data 的起始偏移地址和终止偏移地址。并在该目录下的 vmlinux.lds.S 脚本里定义了.piggydata section, section 内部也定义了一个变量 __piggy_size_addr,这个变量正好指向了 piggy_data 最后 4 个字节。因此在 LC0+32 处定义为“input_data_end - 4”,因此以上数据都可以知道压缩内核解压之后的长度。其他压缩方法同理。接着执行如下命令:

```
/**
 * The kernel build system appends the size of the
 * decompressed kernel at the end of the compressed data
 * in little-endian form.
 */
ldrb    r9, [r10, #0]
ldrb    lr, [r10, #1]
orr      r9, r9, lr, lsl #8
ldrb    lr, [r10, #2]
ldrb    r10, [r10, #3]
orr      r9, r9, lr, lsl #16
orr      r9, r9, r10, lsl #24
```

通过上面的分析可以知道,r10 寄存里存储 piggy_data 的最后面四个字节地址,这个地址存储着压缩内核解压之后的长度,也就是 Image 的长度,但是其长度在这四个字节里按大端格式存储,因此需要上面的代码将大端数据读出转换为小端格式。代码逻辑很简单,就是使用 ldrb 指令从 r10 对应的地址上读一个字节,然后调整字节序,最后压缩内核解压之后的长度存储到 r9 寄存器里。获得了 Image 长度的正确值后,接着继续执行代码:

```

#ifndef CONFIG_ZBOOT_ROM
    /* malloc space is above the relocated stack (64k max) */
    add    sp, sp, r0
    add    r10, sp, #0x10000
#endif
    mov    r5, #0                @ init dtb size to 0

```

这里主要作用是分配 64K 的空间。首先矫正了堆栈的虚拟地址,并且将堆栈之后增加了 64K 空间用于 malloc,将 malloc + stack + zImage 的长度存储到 r10 寄存器中。然后将 r5 设置为 0 供 DTB 使用,但是由于本实践不支持 DTB APPEND 模式,因此接下来执行代码如下:

```

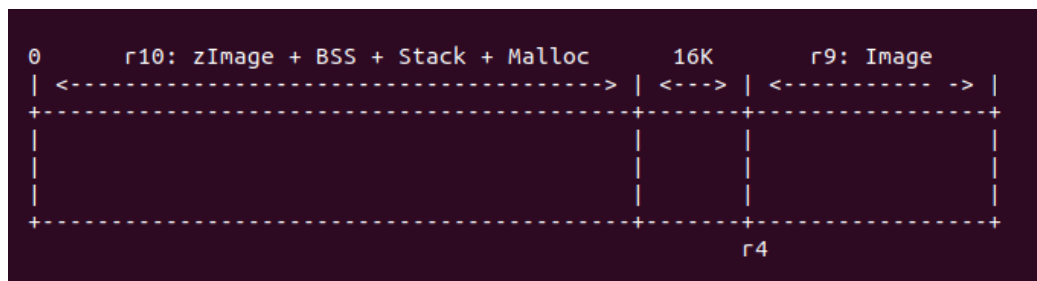
/**
 * Check to see if we will overwrite ourselves.
 *  r4  = final kernel address (possibly with LSB set)
 *  r9  = size of decompressed image
 *  r10 = end of this image, including bss/stack/malloc space if non XIP
 * We basically want:
 *  r4 - 16k page directory >= r10 -> OK
 *  r4 + image length <= address of wont_overwrite -> OK
 * Note: the possible LSB in r4 is harmless here.
 */
    add    r10, r10, #16384
    cmp    r4, r10
    bhs    wont_overwrite
    add    r10, r4, r9
    adr    r9, wont_overwrite
    cmp    r10, r9
    bls    wont_overwrite

```

这段代码的主要任务是确定当前 zImage 运行的范围会不会与内核解压之后的地址范围重合,如果重合,那么 zImage 要做相应的调整,这里涉及到 zImage 重定位到一个新的地址继续运行,这样 zImage 和解压内核相互影响。在执行代码之前,r4 指向解压内核的起始地址,r9 代表解压之后内核的长度,r10 寄存器代表当前 zImage 的长度(该长度也包含了 bss/stack/malloc)的长度,因此需要做对比操作。那么在什么情况下不用重定位或重合呢,如下面几种情况:

$$r4 - 16K \geq r10$$

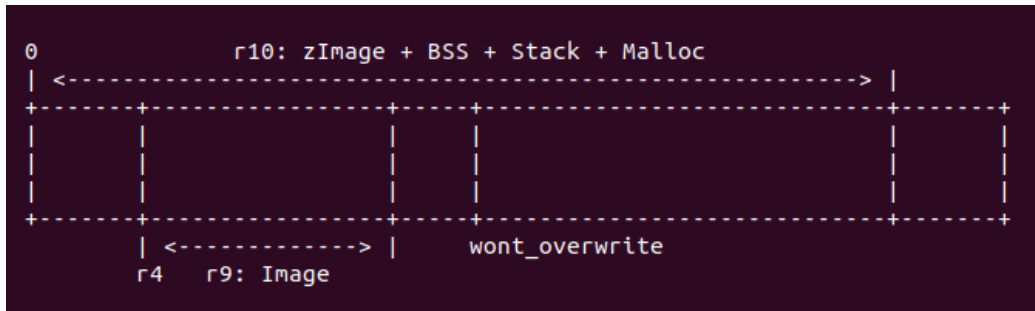
对于这种情况,内存分布如下:



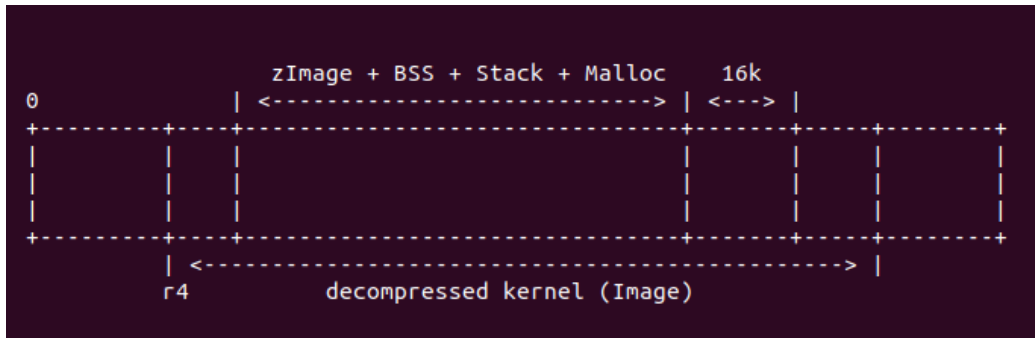
在这种情况下,zImage 运行的地址域与内核解压之后的地址域是不重合的,所以 zImage 不需要重定位,直接在原始地址上直接运行。

$$r4 + image < wont_overwrite$$

对于这种情况,内存布局如下:



首先调用命令“add r10, r10, #16384”, 将 r10 的长度再增加 16K, 此时 r10 代表 zImage 长度加上 BSS/Stack/Malloc, 再加上 16K 的长度, 此时 r10 也可以表示 zImage 运行时完整的长度。接着调用“cmp r4, r10”命令, 查看此时 zImage 的长度域与解压内核长度域之间的关系是否满足“r4 - 16K >= r10”条件, 如果满足, 则执行“bhs wont_overwrite”, 这样 zImage 就不需要重定位; 如果不满足, 那么继续确认是否满足第二个条件, 执行命令“add r10, r4, r9”, 使 r10 寄存器存储解压内核的终止物理地址, 再调用命令“adr r9, wont_overwrite”获得 zImage 中 wont_overwrite 的地址, 最后执行命令“cmp r10, r9”, 如果 r10 小于 r9, 那么满足第二个条件, 则执行“bls wont_overwrite”, zImage 不需要重定位; 反之 zImage 需要重定位。zImage 与解压之后的内核存在重叠时, zImage 需要重定位, 它们之间的关系如下:



经过上面代码执行, r9 寄存器存储着 wont_overwrite 的地址, r10 寄存器存储着解压之后内核的终止地址。接下来运行代码:

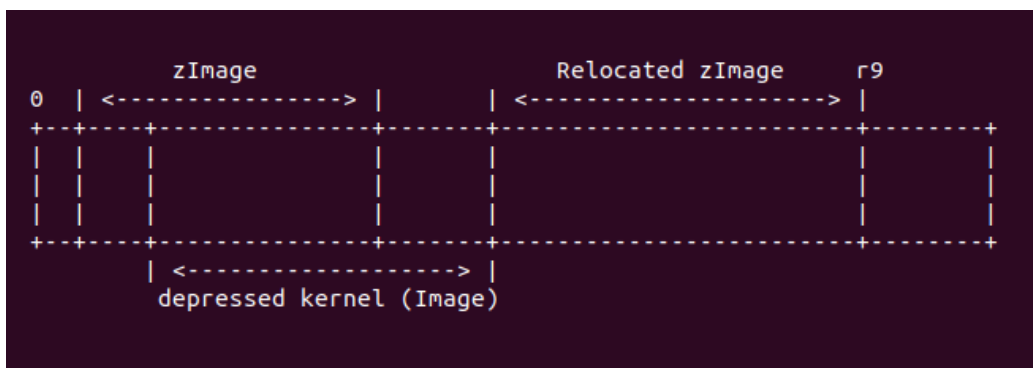
```
/**
 * Relocate ourselves past the end of the decompressed kernel.
 *   r6 = _edata
 *   r10 = end of the decompressed kernel
 * Because we always copy ahead, we need to do it from the end and go
 * backward in case the source and destination overlap.
 */
/**
 * Bump to the next 256-byte boundary with the size of
 * the relocation code added. This avoids overwriting
 * ourself when the offset is small.
 */
add    r10, r10, #((reloc_code_end - restart + 256) & ~255)
bic    r10, r10, #255

/* Get start of code we want to copy and align it down. */
adr    r5, restart
bic    r5, r5, #31
```

此时 r10 寄存器存储着解压之后内核的终止地址。” $((\text{reloc_code_end} - \text{restart} + 256) \& \sim 255)$ ”表示了 head.S 中 reloc_code_end 的长度,并按 256 字节对齐,运行命令之后,r10 寄存器存储了解压之后内核的终止地址再加上 head.s 重定位代码的长度。并将 head.S 中 restart 的地址存储在 r5 寄存器中,并按 32 字节对齐。接下来执行代码:

```
sub    r9, r6, r5    @ size to copy
add    r9, r9, #31   @ rounded up to a multiple
bic    r9, r9, #31   @ ... of 32 bytes
add    r6, r9, r5
add    r9, r9, r10
```

此处,r6 寄存器表示 zImage 不带 BSS 段的长度,即原始 zImage 的长度。这里调用命令“sub r9, r6, r5”,将 zImage 的长度减去需要 head.S 中需要重定位的长度之后的值,存储到 r9 寄存器中,并将 r9 寄存器按 32 字节对齐。接着将 r9 寄存器的值存储到 r6 寄存器中,这样 r6 寄存器存储着 zImage 减去 head.S 中重定位长度之后的终止地址。接着执行命令“add r9, r9, r10”,通过这个命令,r9 存储了 zImage 重定位之后的结束物理地址,他们之间关系如下图:



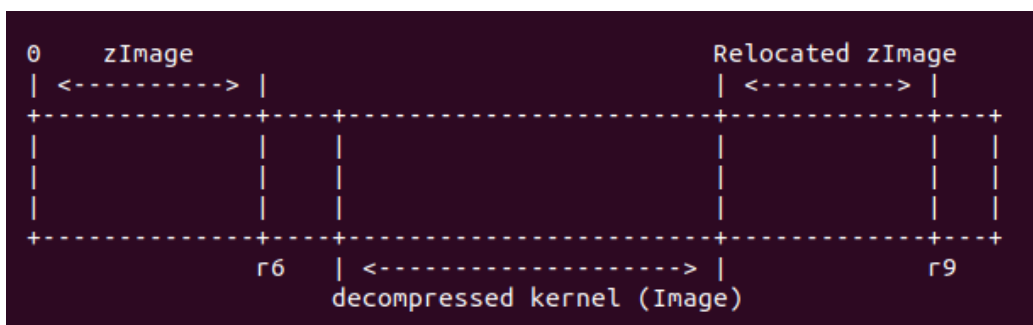
接下来执行代码:

1:

```
ldmdb  r6!, {r0 - r3, r10 - r12, lr}
cmp    r6, r5
stmdb  r9!, {r0 - r3, r10 - r12, lr}
bhi    1b

/* Preserve offset to relocated code. */
sub    r6, r9, r6
```

这段代码的主要任务就是搬运 zImage 到重定位的位置,搬运的内容不包括 zImage 的 BSS/ Stack/ Malloc 区域。此时 r6 寄存器存储着 zImage 的减去 head.S 重定位段之后的结束地址。r9 寄存器存储着解压之后的内核终止地址加上 zImage 重定位的地址。此时内存布局如下:



这里使用 `ldmdb` 从 `r6` 对应的 `zImage` 的末尾往重定位 `zImage` 的末尾拷贝数据，这里也就是搬运 `zImage` 到新的地址。`ldmdb` 一直循环知道 `r6` 的地址与 `r5` 对应的地址重合，方才停止循环。接下来执行的代码如下：

```
#ifndef CONFIG_ZBOOT_ROM
    /* cache_clean_flush may use the stack, so relocated it */
    add    sp, sp, r6
#endif
```

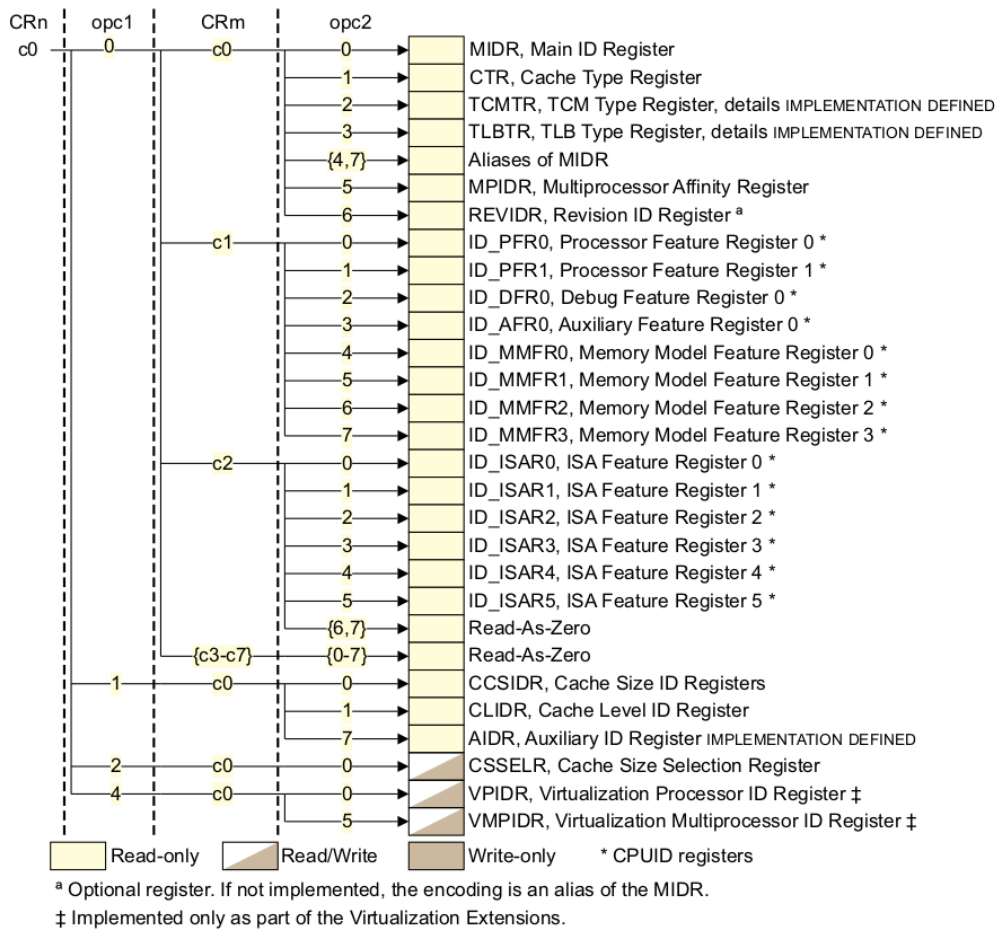
并未定义 `CONFIG_ZBOOT_ROM` 宏情况下，由于接下来要执行 `cache_clean_flush` 需要使用堆栈，所以将堆栈指向一个合适的位置，这里将堆栈加上 `r6` 寄存器的值。接下来调用 `cache_clean_flush`。具体代码如下：

```
/**
 * Clean and flush the cache to maintain consistency
 *
 * On exit,
 * r1, r2, r3, r9, r10, r11, r12 corrupted
 * This routine must preserve:
 * r4, r6, r7, r8
 */
    .align 5
cache_clean_flush:
    mov    r3, #16
    b      call_cache_fn
```

`cache_clean_flush` 的定义很简单，与 `cache_on` 一样的机制，都是在 `CACHE` 表中找到 `__armv7_mmu_cache_flush` 的入口，具体查找过程，查看前面关于 `call_cache_fn` 源码解析，通过 `call_cache_fn` 之后，会定位到 `__armv7_mmu_cache_flush` 处，代码如下：

```
__armv7_mmu_cache_flush:
    tst    r4, #1
    bne    iflush
    mrc    p15, 0, r10, c0, c1, 5 @ read ID_MMFR1
    tst    r10, #0xf << 16 @ hierarchical cache (ARMv7)
    mov    r10, #0
    beq    hierarchical
    mcr    p15, 0, r10, c7, c14, 0 @ clean+invalidate D
    b      iflush
```

首先检查 `r4` 寄存器的最低位是否置位，从前面的代码可以知道，`MMU` 打开的情况下，`r4` 最低位清零；如果 `MMU` 未启用，那么 `r4` 最低位置位。由之前的分析可知，此处 `MMU` 已经启用，所以 `tst` 的结果未零，那么“`bne iflush`”命令将不被执行。接下来执行的代码是：“`mrc p15, 0, r10, c0, c1, 5`”，此时 `CP15 C0` 寄存器的布局如下：



通过上面的代码选中了 ID_MMFR1 寄存器,该寄存器用于指定当前 CACHE 的类型,其位图如下:

The ID_MMFR1 bit assignments are:

31	28	27	24	23	20	19	16	15	12	11	8	7	4	3	0
Branch predictor	L1 cache test and clean	L1 unified cache	L1 Harvard cache	L1 unified cache set/way	L1 Harvard cache set/way	L1 unified cache VA	L1 Harvard cache VA								

接着调用 tst 命令查看 ID_MMFR1 寄存器的 [19:16] 域,该域表示:

L1 Harvard cache, bits[19:16]

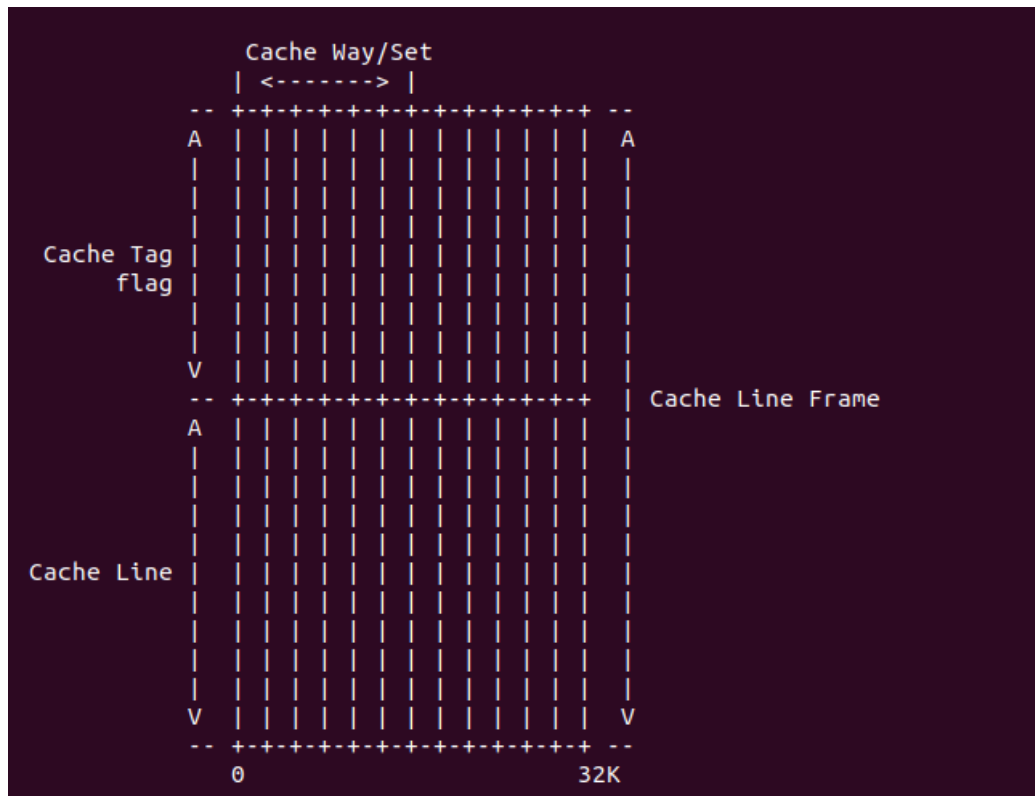
Indicates the supported entire Level 1 cache maintenance operations, for a Harvard cache implementation. Permitted values are:

- 0b0000 None supported. This is the required setting for ARMv7, because ARMv7 requires a hierarchical cache implementation.
- 0b0001 Supported entire Level 1 cache operations are:
 - Invalidate instruction cache, including branch predictor if appropriate
 - Invalidate branch predictor, if appropriate.
- 0b0010 As for 0b0001, and adds:
 - Invalidate data cache
 - Invalidate data cache and instruction cache, including branch predictor if appropriate.
- 0b0011 As for 0b0010, and adds:
 - Clean data cache. Uses a recursive model, using the cache dirty status bit.
 - Clean and invalidate data cache. Uses a recursive model, using the cache dirty status bit.

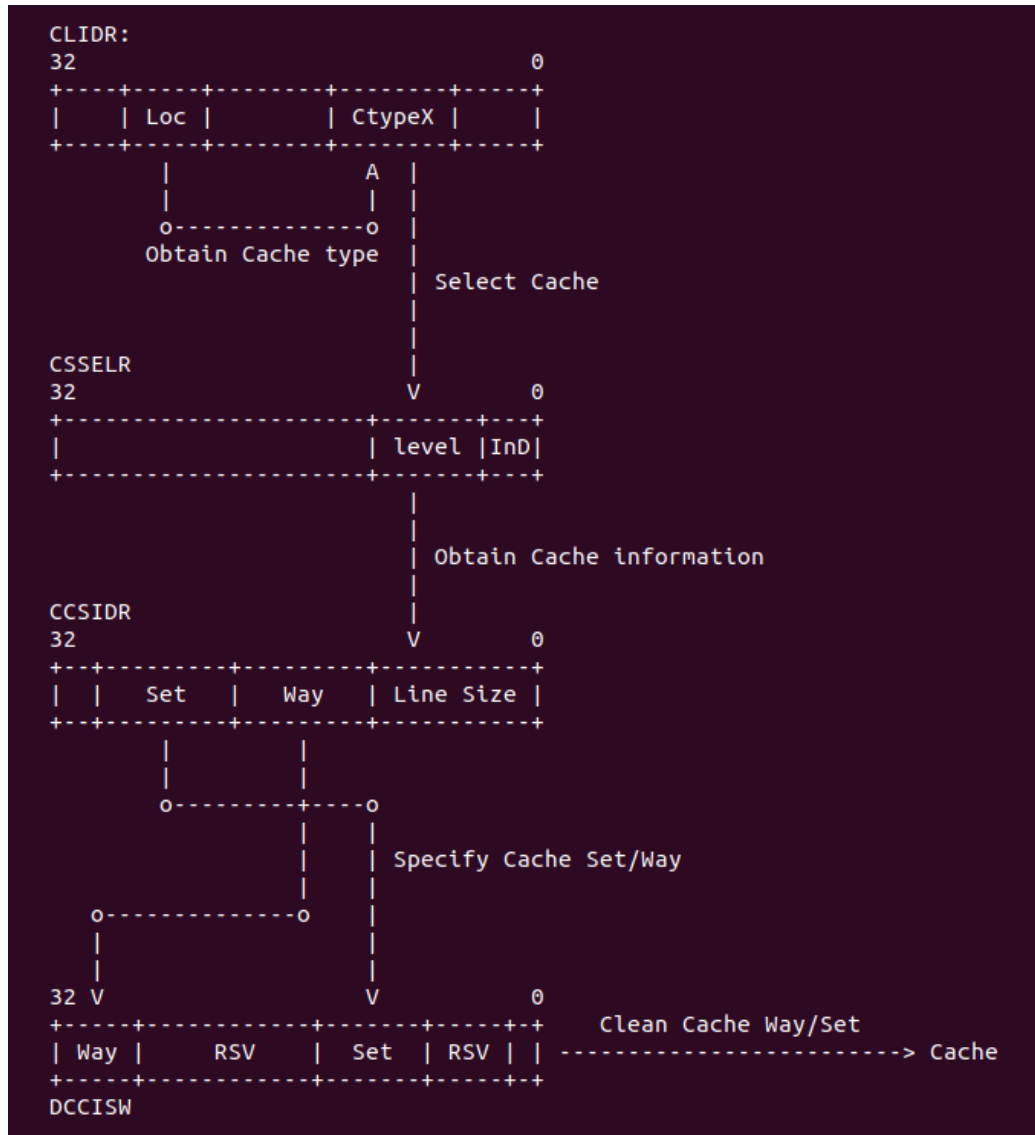
If this field is set to a value other than 0b0000 then the L1 unified cache field, bits[23:20], must be set to 0b0000.

有上面的定义可知,该域用于指示 L1 cache 的维护操作。但在 armv7 中,这是不需要的,因此 tst 的结果等于 0,那么代码接下来跳转到 hierarchical 处继续执行。跳转到 hierarchical 处继续执行,由于 armv7 的 L1 cache 按分层管理,要给 cache 进行 flush 操作,首先要了解一下 cache 的基础知识,这里只介绍必要的知识,更多 cache 知识请查看:[ARMv7-L1 cache manual](#)

L1 cache: 在 ARMv7 中,cache 分为了指令 cache (I-Cache) 和数据 cache (D-cache), cache 是用来加速内存访问,其逻辑结构如下图:



cache 的最小数据单位是 cache line, cache line 的长度成为 cache size; 为了标记每个 cache line, 使用的标记叫 cache tag, cache tag、cache line 与一些标志信息组成一个 cache line frame; 多个 cache line frame 组成一个 cache set; cache 被分成多个 cache set, 每个 cache set 含有的 cache line 数成为 cache way. 在 armv7 中要 flush D-cache 按如下逻辑:



在 armv7 中,首先从 CLIDR 寄存器中读取 LoC 域对应的 Ctype,从中找到 D-cache,然后在 CSSELR 寄存器写入 D-cache 的 level 信息之后,CCSIDR 寄存器就指向了所要查找 cache 的信息,这些信息存储在 CCSIDR 寄存器中,从这个寄存器中可以获得当前 CACHE 的 cache line 数值,cache Set/Way 的值,通过这些值可以计算出需要 flush 的长度,最后将要清楚的值写入到 DCCISW 寄存器中,这样该 level 的 cache 就被 flush 完了,接着遍历 flush 下一级 cache。因此 CACHE 的 flush 逻辑就是这样。接下来通过代码分析具体过程:

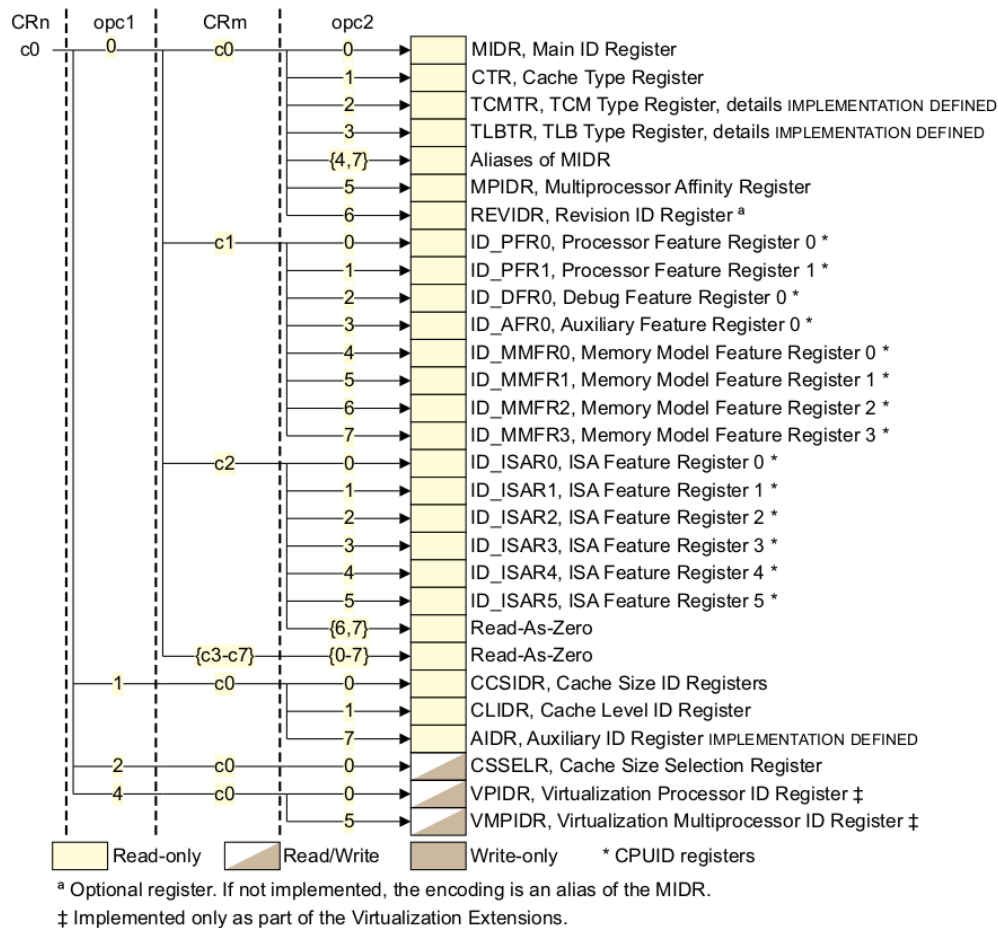
hierarchical:

```

mcr      p15, 0, r10, c7, c10, 5 @ DMB
stmfd    sp!, {r0-r7, r9-r11}
mrc      p15, 1, r0, c0, c0, 1   @ read clidr
ands     r3, r0, #0x7000000      @ extract loc from clidr
mov      r3, r3, lsr #23         @ left align loc bit field
beq      finished                @ if loc is 0, then no need to c
mov      r10, #0                 @ start clean at cache level 0

```

首先做一次 DMB 内存屏蔽操作,将之前的内存访问都同步。然后使用 stmfd 指令保存指定的寄存器,接着调用 mrc 寄存器读取 CP15 C0 寄存器,选择如下:



通过上面选中了 CLIDR 寄存器,该寄存器的布局如下图:



然后执行代码“ands r3, r0, #0x7000000”, 以此读取 CLIDR [26:24] 域, 这个域用于 LoC (Level of Coherence for the cache hierarchy). 这个域用于指定一致性数据 cache 所在的 cache Level, 然后通过“mov r3, r3, lsr #23”获得 cache level 的具体数值。如果该值为 0, 那么代表没有对应的 cache, 直接跳转到 finished 处; 如果该值为 0, 那么代码存在对应需要 flush 的 cache, 那么继续执行下面的代码不跳转。

loop1:

```

add    r2, r10, r10, lsr #1    @ work out 3x current cache leve
mov    r1, r0, lsr r2          @ extract cache type bits from c
and    r1, r1, #7              @ mask of the bits for current c
cmp    r1, #2                  @ see what cache we have at this
blt    skip                    @ skip if no cache, or just i-ca

```

```

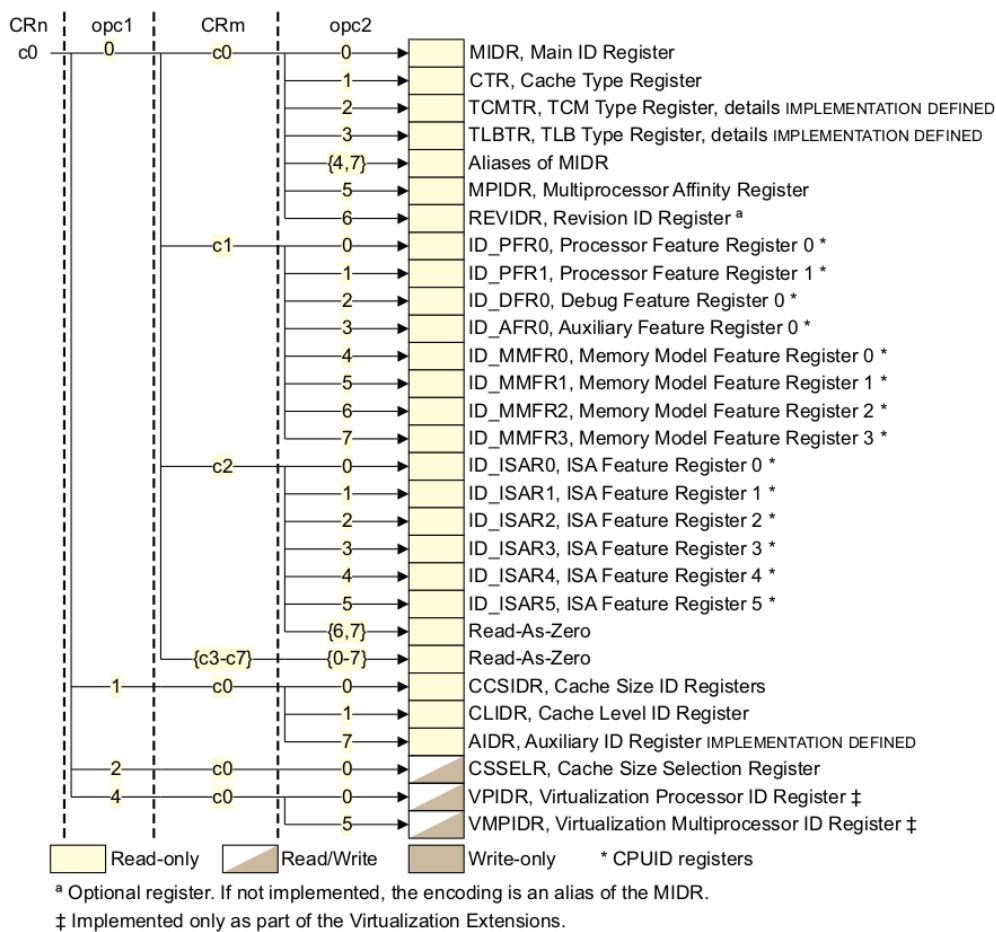
mcr    p15, 2, r10, c0, c0, 0 @ select current cache level in
mcr    p15, 0, r10, c7, c5, 4 @ isb to sych the new cssr&csidr
mcr    p15, 1, r1, c0, c0, 0 @ read the new csidr

```

由于在 armv7 中 LoC cache 采用分级模式,所以在 flush cache 的时候,需要从 ctype0 开始 flush,每个 cache ctype 之间相差 3,所以这里使用“add r2, r10, r10, lsr #1”达到遍历下一个 cache 的作用。接着就是对 ctypes 对应的值进行对比,这个值从手册中可以知道:1)当该值为 0,代表没有 cache。2)该值为 1 代表指令 cache。3)该值为 2 代表数据 cache。4)该值为 3 代表指令和数据分离的 cache。5)该值为 4 代表 Unified cache。

1. 其他值为 Reserved。如果此时“cmp r1, #2”的值小于 2,那么就代表这个

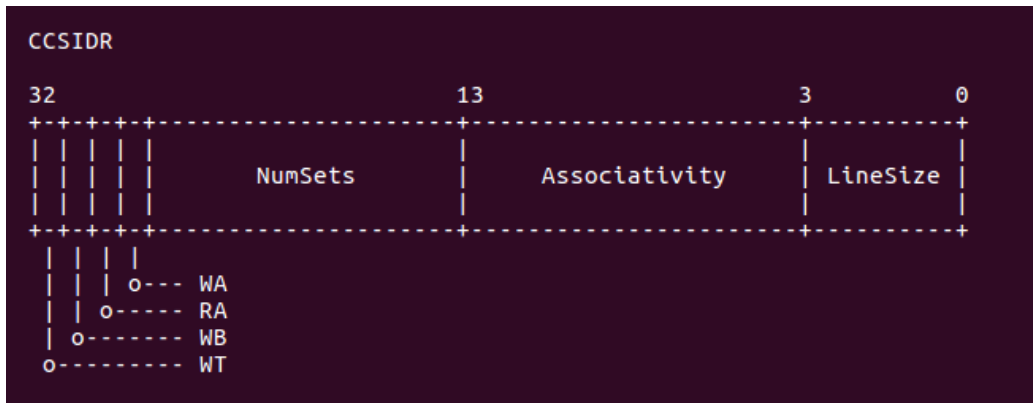
cache 是一个指令 cache 或者没有 cache。如果 r1 寄存器的值小于 2 那么直接跳转到 skip 处继续执行,如果大于等于 2,那么继续往下执行。接着如果是数据 cache,那么调用命令“mcr p15, 2, r10, c0, c0, 0”,此时 CP15 C0 的布局如下:



选中的寄存器是 CSSELR 寄存器,其内存布局如下:



这里用于选择 Cache 的级数,在 armv7 中,只要在 CSSELR 寄存器的 Level 域中选中了 cache Level,那么 CSIDR 寄存器就反应被选中 cache 相关信息。为了是 CSSELR 写入有效,这里调用了内存屏蔽指令 ISB,执行完 ISB 之后,CSIDR 寄存器的值就是反应 CSSELR 选中的结果,CSSELR 的内存布局如下:



正如上图所示,CSIDR 寄存器包含了当前 cache 的 Set 域,Way 域,以及 line size 域。开发者可以在上面代码中加入适当的断点,然后使用 GDB 进行调试,调试情况如下:

```
(gdb) b BS_debug
Breakpoint 1 at 0x600102a4: file arch/arm/boot/compressed/head.S, line 341.
(gdb) c
Continuing.
Breakpoint 1, BS_debug () at arch/arm/boot/compressed/head.S:341
341  add    r2, r10, r10, lsr #1    @ work out 3x current cache level
(gdb) n
342  mov    r1, r0, lsr r2    @ extract cache type bits from clidr
(gdb) info reg r10
r10    0x0    0
(gdb) info reg r2
r2     0x0    0
(gdb) n
343  and    r1, r1, #7    @ mask of the bits for current cache only
(gdb) info reg r1
r1     0x9000003  150994947
(gdb) n
344  cmp    r1, #2        @ see what cache we have at this level
(gdb) info reg r1
r1     0x3     3
(gdb) n
345  blt    skip    @ skip if no cache, or just i-cache
(gdb) n
346  mcr    p15, 2, r10, c0, c0, 0    @ select current cache level in cssr
(gdb) n
347  mcr    p15, 0, r10, c7, c5, 4    @ isb to sych the new cssr&csidr
(gdb) info reg r10
r10    0x0    0
(gdb) n
348  mrc    p15, 1, r1, c0, c0, 0    @ read the new csidr
(gdb) n
```

```

349  and    r2, r1, #7           @ extract the length of the cache lines
(gdb) info reg r1
r1    0xe00fe019                -535830503

```

从上面的实践可以看出, LoC 所使用的 cache 是一个数据 cache, 对应的 Ctype1, 并且 Ctype0 的 cache 为 0x3, 将该值写入到 CSIDR 寄存器, 就获得 Ctype1 对应的 cache 长度信息。从 r1 读入的值为 0xe00fe019, 从这个数值中可以知道 Cache 的 set 数为 $(0x3F + 1)$, 也就是该 cache 包含了 128 个 set。每个 set 包含的 cache line 数 $(0x3 + 1)$, 那么该 cache 是 4 way。cache line size 是 8 字节。接下来执行的代码是:

```

and    r2, r1, #7           @ extract the length of the cache lines
add    r2, r2, #4           @ add 4 (line length offset)
ldr    r4, =0x3ff
ands   r4, r4, r1, lsr #3    @ find maximum number on the way size
clz    r5, r4               @ find bit position of way size increment
ldr    r7, =0x7fff
ands   r7, r7, r1, lsr #13   @ extract max number of the index size

```

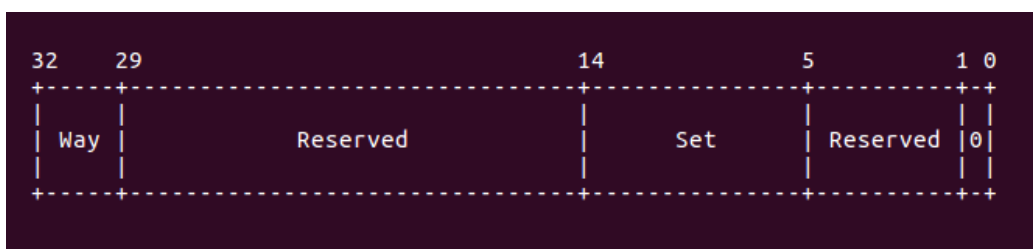
从之前的分析可以知道, r1 寄存器里面存储着当前 cache 长度相关的信息, 首先使用 “and r2, r1, #7” 获得 cache line 的长度。然后调用 “add r2, r2, #4”, 这条命令的作用是为了后面 cache flush 特定寄存器写信息的特殊格式要求, 可以参看后面 flush 操作。接着将常量 0x3ff 赋值给 r4, 然后执行命令 “ands r4, r4, r1, lsr #3”, 通过这条命令读出 cache way 的信息, 接着调用 “clz r5, r4” 也是为了拼凑一个格式化数据。同理, 将常量 0x7fff 赋值给 r7, 然后获得 cache 的 set 信息。接下来执行的代码如下:

```

loop2:
    mov    r9, r4           @ create working copy of max way size
loop3:
    ARM(orr r11, r10, r9, lsl r5) @ factor way and cache number into r11
    ARM(orr r11, r11, r7, lsl r2) @ factor index number into r11
    mcr    p15, 0, r11, c7, c14, 2 @ clean & invalidate by set/way
    subs   r9, r9, #1       @ decrement the way
    bge    loop3
    subs   r7, r7, #1       @ decrement the index
    bge    loop2
skip:
    add    r10, r10, #2 @ increment cache number
    cmp    r3, r10
    bgt    loop1

```

这段代码就是按 set/way 方式 flush cache, 这讲解代码之前, 先了解 cache flush 的方法, armv7 中采用 set/way 方式管理 cache。通过之前 cache 基本原理可以知道, 每个 cache set 包含了 way 个 cache line, 而 cache 被均分成多个 cache set。因此可以将 cache set 理解为行, cache way 理解为列, 所以刷新的时候按行-列的模式刷, 因此这里的代码就是用于实现将所有 set 和 way 都 flush。另外, armv7 中, 往特定寄存器写入 set/way 信息就可以刷新指定的 set/way cache, 这个寄存器就是 DCCISW 寄存器, 这个寄存器的位图如下:



从上图可以知道,要刷新特定的 cache line,那么需要往 DCCISW 寄存器内写入 Way 和 Set 的信息。因此在分析之前的代码,为什么 line size 的值要加上 4,以及 way 的值要这么处理也就知道了,就是为了制作出一个符合 DCCISW 格式的数据。因此上面的代码分析如下:首先调用“mov r9, r4”,为了进行 set 的遍历,接着“orr r11, r10, r9, lsl r5”和“orr r11, r11, r7, lsl r2”命令也是为了构建 DCCISW 格式化数据,准备好数据之后,就将数据写入到 DCCISW 寄存器里面,使用的命令是“mcr p15, 0, r11, c7, c14, 2”。接下来的代码就是遍历所有的 set 以及 set 里面的所有 way,通过上面的操作,当前 cache 的所有 set/way 就被 flush 完毕。最后到达 skip 处,如果当前 cache flush 完毕,那么跳转到 loop1 采用同样的方式遍历下一个 cache。接下来执行的代码是:

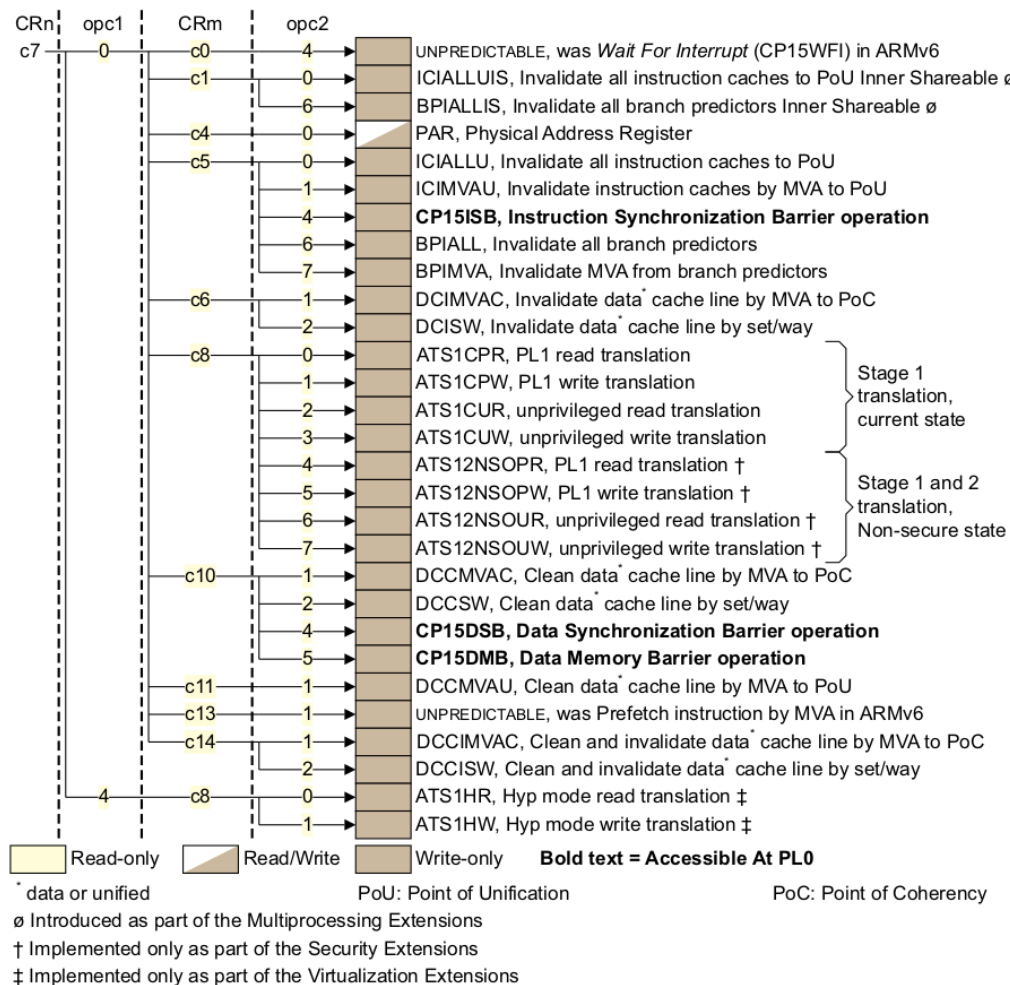
finished:

```
ldmfd    sp!, {r0-r7, r9-r11}
mov      r10, #0                @ switch back to cache level 0
mcr      p15, 2, r10, c0, c0, 0 @ select current cache level in cssr
```

iflush:

```
mcr      p15, 0, r10, c7, c10, 4 @ DSB
mcr      p15, 0, r10, c7, c5, 0  @ invalidate I+BTB
mcr      p15, 0, r10, c7, c10, 4 @ DSB
mcr      p15, 0, r10, c7, c5, 4  @ ISB
mov      pc, lr
```

cache flush 完毕之后,就做最后的收尾工作,将期间使用过的寄存器都恢复原值,然后将当前 cache 设置为 0。然后刷新 I-cache。首先调用 DSB 内存屏障,将之访问全部落盘,然后调用“mcr p15, 0, r10, c7, c5, 0”,此时 CP15 C7 的布局如下:



通过上面的命令选中了寄存器:ICIALLU,向该寄存器写入任意值就会让 I-cache 无效。接着调用

两条内存屏障指令 DSB 和 ISB, 让所有改变都生效。最后将 lr 赋值给 pc, 实现调用返回。支持 armv7 的 flush 操作已经全部完成。代码继续执行如下:

```
badr    r0, restart
add     r0, r0, r6
mov     pc, r0
```

首先获得 restart 的地址, 存储到 r0 寄存器内, 然后将 r0 寄存器的值加上 r6 偏移值, 以此计算出 restart 重定位之后的地址, 最后将该值赋值给 pc, 然后 CPU 就跳转到 restart 重定位处继续执行。

6 tools

- config: 多种 config 前端见表格2。

表 2: Config 前端

Name	Description
config	Update config using a line-oriented program
menuconfig	Update config using a menu-based program
xconfig	Update config using a QT-based front end
gconfig	Update config using a GTK-based front end
oldconfig	Update config using a provided .config as the base
randconfig	New config with random answer to all options
defconfig	New config with default answer to all options
allmodconfig	New config that selects modules, when possible
allyesconfig	New config in which all options are accepted with yes
allnoconfig	New minimal config

7 SOC 架构

该 SOC 集成了 2 个 NODE, 每个 NODE 集成 2 个 Cluster, 每个 Cluster 集成 2 个物理 Core, 每物理 Core 虚拟出 2 个逻辑 CPU

- SOC
 - Node 0
 1. Cluster 0
 - (a) Core 0 a. CPU 0 b. CPU 1
 - (b) Core 1 a. CPU 2 b. CPU 3
 2. Cluster 1
 - (a) Core 2 a. CPU 4 b. CPU 5
 - (b) Core 3 a. CPU 6 b. CPU 7
 - Node 1
 1. Cluster 2
 - (a) Core 4 a. CPU 8 b. CPU 9
 - (b) Core 5 a. CPU 10 b. CPU 11
 2. Cluster 3
 - (a) Core 0 a. CPU 12 b. CPU 13
 - (b) Core 7 a. CPU 14 b. CPU 15

7.1 MMU

ARMv8 的 Page Table 结构如图9所示。其分为 4 级，每级页表有 512 个条目，支持如下 3 种表条目描述符。

- Table descriptor, 指向下一级 table;
- Page descriptor, 指向一个 4KB(或 64KB、1MB)大小 page size 的页面;
- Block descriptor, 指向一个 block size 的内存区域; 在 1 级页表中 block size 是 1GB, 2 级页表中 block size 是 2MB。

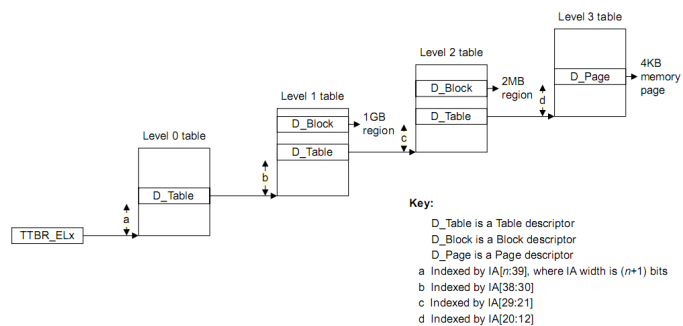


图 9: ARMv8 Page Table